

A COOKBOOK FOR NODE.JS DEVELOPERS

PHP & Laravel Backend Cookbook

Learn modern PHP 8.3+ and Laravel by mapping every concept back to the Node.js you already know.

PHP 8.3+

Laravel 11/12

Eloquent

Sanctum

Queues & Horizon

Octane

Pest

Written for engineers fluent in JavaScript / TypeScript, Node.js, Express, Prisma and REST. Every chapter answers: what do I do in Node, how is it done here, why is it different, when to use which.

Edition 2026 · Node.js → PHP → Laravel

Table of Contents

How to Use This Book	6
The one idea that explains PHP	6
How the two frameworks map	7
The Learning Roadmap	7
Part 1 • The PHP Language	9
1.1 Variables, Constants, and Scope	9
1.2 Scalar Types: strings, numbers, booleans, null	10
1.3 Arrays and Associative Arrays	11
1.4 Comparison and Truthiness	12
1.5 Conditionals, Loops, and match	13
1.6 Functions, Defaults, Named & Variadic Arguments	14
1.7 Closures, Arrow Functions, and Callbacks	14
1.8 References	15
1.9 Objects, Classes, and Constructor Promotion	15
1.10 Interfaces, Abstract Classes, Traits, Enums	16
1.11 Type Declarations: union, nullable, intersection	18
1.12 Namespaces, Autoloading, Exceptions, Generators, Attributes	18
Part 2 • Functional Programming in PHP	20
2.1 First-Class Functions and Higher-Order Functions	20
2.2 map / filter / reduce — the argument-order trap	20
2.3 Why raw PHP FP gets ugly — and the fix	21
2.4 The Collection pipeline vocabulary	22
2.5 Currying, Partial Application, Composition, Pipe	23
2.6 Lazy Evaluation: Generators and LazyCollection	24
Part 3 • The Node Event Loop vs the PHP Runtime	26
3.1 The Node.js model, restated precisely	26
3.2 The classic PHP model: shared-nothing, one request per process	27
3.3 Why this is often simpler, not just different	27
3.4 OPcache and the bootstrap cost	28
3.5 Blocking I/O and where concurrency comes from	28
3.6 Moving work off the request: queues, workers, scheduler	29
3.7 Laravel Octane: giving PHP a Node-like runtime	30
Part 4 • Composer — PHP's npm	32
4.1 The direct translation	32
4.2 Version constraints and semantic versioning	32
4.3 Autoloading and PSR-4 — the part with no npm analog	33
4.4 Scripts and package development	34
Part 5 • Laravel Architecture for Express Developers	35
5.1 Project structure	35
5.2 The request lifecycle	35
5.3 The Service Container and Dependency Injection	36
5.4 Service Providers — the bootstrap layer	37

5.5 Facades and Contracts — the "magic" demystified	38
Part 6 · Routing	40
6.1 The basics, side by side	40
6.2 Parameters, constraints, and query strings	41
6.3 Route groups: shared prefix, middleware, and naming	41
6.4 Route Model Binding — the feature you'll miss most going back	42
6.5 Resource routes — CRUD in one line	43
6.6 Route caching and listing	43
Part 7 · Database & Eloquent (vs Prisma)	45
7.1 The core philosophical difference	45
7.2 Migrations — schema as versioned code	45
7.3 Models, mass assignment, casts, and enums	46
7.4 CRUD with the query builder and Eloquent	47
7.5 Relationships	48
7.6 Eager loading and the N+1 problem	49
7.7 Scopes, accessors, and mutators	50
7.8 Pagination, transactions, and locking	51
Part 8 · A Complete CRUD Feature	53
8.1 The anatomy of a Laravel CRUD feature	53
8.2 Validation: Form Requests (your Zod/Joi equivalent)	53
8.3 Response shaping: API Resources (your serializer)	54
8.4 The controller: thin orchestration	55
8.5 Factories and seeders — fake data as first-class	56
Part 9 · Production REST APIs	57
9.1 Status codes and HTTP verbs — the contract	57
9.2 Consistent error responses	57
9.3 Pagination, filtering, sorting, searching	58
9.4 API versioning	59
9.5 Idempotency and idempotency keys	59
9.6 Webhooks (receiving)	60
Part 10 · Authentication & Authorization	61
10.1 Password hashing	61
10.2 Two auth worlds: sessions vs API tokens	61
10.3 Sanctum API tokens in practice	62
10.4 Authorization: Gates and Policies	62
10.5 Roles and permissions at scale.	63
10.6 Email verification & password reset	63
Part 11 · Security	65
11.1 SQL injection	65
11.2 Mass assignment.	65
11.3 CSRF and CORS	66
11.4 XSS and output encoding	66
11.5 Rate limiting	66
11.6 Encryption, secrets, and signed URLs	66
11.7 File upload security.	67

11.8 Webhook signature verification	67
11.9 The pre-flight security checklist	67
Part 12 • Application Architecture	69
12.1 The default: fat where it belongs, thin everywhere else	69
12.2 Services and Actions — when logic outgrows the controller	69
12.3 DTOs and Value Objects	70
12.4 Events and Listeners — decoupling side effects	71
12.5 Repositories — usually unnecessary here	71
12.6 A pragmatic layering rule of thumb	72
Part 13 • Queues & Background Jobs	73
13.1 The mental map	73
13.2 A job, dispatched and handled	73
13.3 Running workers (and why they are different from FPM).	74
13.4 Retries, backoff, failures, idempotency.	74
13.5 Chains, batches, and delays	75
13.6 Monitoring with Horizon	75
Part 14 • Task Scheduling	77
14.1 One cron entry to rule them all	77
14.2 The frequency vocabulary	77
14.3 Production-critical guards	78
Part 15 • Caching & Redis	80
15.1 The cache API	80
15.2 Cache-aside with remember — the pattern you'll use most	80
15.3 Invalidation strategies	81
15.4 Atomic locks and distributed locks	81
15.5 Cache stampede (the thundering herd)	82
Part 16 • HTTP Clients	83
16.1 The basics, side by side	83
16.2 Response handling and error semantics	84
16.3 Retries, backoff, and timeouts	84
16.4 Concurrent requests	84
16.5 Testing without hitting the network	85
Part 17 • Files & Object Storage (S3)	86
17.1 The disk abstraction	86
17.2 Handling uploads safely	86
17.3 Private files and temporary (signed) URLs	87
17.4 Streaming and large files.	87
17.5 Image and file processing	88
Part 18 • Testing	89
18.1 The tooling map	89
18.2 Feature tests: the highest-value tests you can write.	89
18.3 Faking the outside world	90
18.4 Unit tests, mocking, and time	91
18.5 What to test, and how much	91

Part 19 • Artisan — The CLI.	93
19.1 The commands you'll live in	93
19.2 Tinker — the REPL	93
19.3 Writing custom commands	94
19.4 The Process API — running external commands	95
Part 20 • Production Deployment	96
20.1 The production topology	96
20.2 The Nginx + PHP-FPM front.	96
20.3 Keeping workers alive with Supervisor	97
20.4 The deploy sequence (zero-downtime).	97
20.5 Config, secrets, and health checks	98
20.6 Docker	98
Part 21 • Advanced Topics	100
21.1 Real-time: broadcasting and WebSockets	100
21.2 Full-text search: Scout	101
21.3 Concurrency and Octane, consolidated	101
21.4 Performance: the measurable wins, in order	102
Part 22 • Capstone Projects	104
22.1 Project A — Task Management API	104
22.2 Project B — E-commerce API	105
22.3 Project C — Webhook / Integration Platform	107
Part 23 • Master Cheat Sheet & Closing Roadmap	110
23.1 Tooling & ecosystem	110
23.2 Language constructs	110
23.3 Framework & HTTP	111
23.4 Data, jobs, and infrastructure	112
23.5 The runtime, in one table	113
23.6 Closing Roadmap — from here to fluent	113

How to Use This Book

You are already a productive backend engineer. You know JavaScript and TypeScript deeply, you ship Node.js and Express services, you model data with Prisma, you think in `async/await`, Promises, and functional pipelines. This book does **not** try to teach you programming. It teaches you a *second ecosystem* by continuously translating it into the one already wired into your head.

Every substantial concept in this book follows the same rhythm:

1. **Concept** — what problem are we solving.
2. **Node.js equivalent** — how you already do it.
3. **PHP / Laravel implementation** — how it is done here.
4. **Side-by-side code** — the two, next to each other.
5. **Important differences** — where the mental model breaks.
6. **Common mistakes** — what trips up Node developers specifically.
7. **Production advice** — what matters when it is live.
8. **Exercise** — a small task to make it stick.

Not every section needs all eight beats, but the spine is always the same question set: *What would I do in Node? How is it done here? Why is it different? When should I use each approach?*

The one idea that explains PHP

If you remember a single thing from this book, remember this: **Node is a long-lived process; classic PHP is not.** A Node server boots once and then handles thousands of requests inside one process with a shared event loop and shared memory. A classic PHP application boots, handles **one** request, and tears everything down — fresh memory, fresh globals, nothing shared — then does it all again for the next request. This "shared-nothing" model is the root cause of almost every difference you will notice: why PHP has no built-in event loop, why global state feels safe, why "just cache it in a module-level variable" does not work the way it does in Express, why queues and workers exist, and why tools like Laravel Octane are a big deal. Keep this contrast in your pocket; we will return to it constantly.

NODE.JS MENTAL MODEL

Express: `app.listen(3000)` runs forever; a memory leak compounds over days. Classic PHP: each request is a brand-new program run. A memory leak barely matters because the process dies in milliseconds — but a slow bootstrap is paid on *every* request. Different runtime, different worries.

How the two frameworks map

Laravel is a batteries-included, convention-over-configuration framework. In the Node world its closest cousin is **NestJS** (structured, DI-driven, opinionated) crossed with **Rails**. It is *not* like bare Express, where you assemble everything yourself. If you have only ever used Express, expect Laravel to feel like it does a lot of "magic" — this book's job is to make every piece of that magic explicit.

The Learning Roadmap

Work the levels in order. Each builds on the last. For every level you get: what to read, what to practice, and what you should be able to build afterward.

Level 1 — Language fundamentals (Parts 1)

Read: Part 1. *Practice:* port 15–20 small Node utility functions to PHP — string manipulation, array reshaping, a small class hierarchy. *You can build:* standalone PHP scripts and library classes with correct types.

Level 2 — Functional programming (Part 2)

Read: Part 2. *Practice:* rewrite a data-transformation module from a Node project using `array_map` / `array_filter` / `array_reduce`, then again with Laravel Collections. *You can build:* clean data pipelines without reaching for loops by reflex.

Level 3 — Runtime & concurrency (Part 3)

Read: Part 3. *Practice:* diagram the request lifecycle of one of your Express apps and its PHP-FPM equivalent; run a blocking vs. queued task experiment. *You can build:* a correct mental model of *where work happens* — inline, queued, or scheduled.

Level 4 — Framework fundamentals (Parts 4–6)

Read: Composer, Laravel architecture, routing. *Practice:* scaffold a Laravel app, wire routes and controllers, add middleware. *You can build:* a routed API skeleton with dependency injection.

Level 5 — Database (Parts 7–8)

Read: Eloquent and CRUD. *Practice:* model Users/Projects/Tasks with relationships, migrations, factories; build full CRUD. *You can build:* a relational data layer with no N+1 problems.

Level 6 — REST APIs (Part 9)

Read: REST APIs. *Practice:* add pagination, filtering, sorting, versioning, and consistent error envelopes. *You can build:* a production-shaped JSON API.

Level 7 — Authentication & security (Parts 10-11)

Read: auth and security. *Practice:* add Sanctum tokens, policies, rate limiting; run through the security checklist. *You can build:* an API safe to expose to the public internet.

Level 8 — Background jobs & scheduling (Parts 13-14)

Read: queues and scheduler. *Practice:* move email sending and a webhook call off the request path; add a nightly scheduled command. *You can build:* responsive endpoints that offload slow work.

Level 9 — Testing (Part 18)

Read: testing. *Practice:* cover your CRUD and auth with Pest feature tests and factories. *You can build:* a suite that lets you refactor fearlessly.

Level 10 — Production deployment (Part 20)

Read: deployment. *Practice:* containerize the app with Nginx + PHP-FPM + Postgres + Redis + a queue worker under Supervisor. *You can build:* a deployable, observable service with zero-downtime migrations.

Level 11 — Advanced architecture (Parts 12, 15-17, 19, 21)

Read: architecture, cache, HTTP clients, files/S3, Artisan, advanced. *Practice:* introduce services/actions/events where they pay off — and delete them where they do not. *You can build:* a maintainable codebase that scales with the team.

Level 12 — Capstone (Part 22)

Read + build: the three capstone projects. *You can build:* production backends end to end, independently.

At the very end you will find a **master cheat sheet** mapping Node → PHP → Laravel across every subsystem — bookmark it.

Part 1 · The PHP Language

PHP and JavaScript grew up in the same era solving the same problem — making web pages dynamic — yet they made almost opposite runtime choices. Modern PHP (8.3+) is a strict, expressive, statically-typeable language that looks nothing like the PHP 5 you may remember. This part maps its syntax and semantics onto the JavaScript and TypeScript you already write.

Every PHP file that only contains code should open with two lines and *no* closing tag:

PHP

```
<?php
declare(strict_types=1);
```

`declare(strict_types=1)` is the single most important line for a TypeScript developer. Without it, PHP coerces `"3"` into `3` at function boundaries like loose JavaScript. With it, passing a string where an `int` is declared throws a `TypeError`. Treat it as mandatory, like `"strict": true` in `tsconfig.json`.

1.1 Variables, Constants, and Scope

Variables start with `$`. There is no `let/const/var` keyword — the `$` is the declaration. Assignment creates the variable.

JAVASCRIPT / NODE.JS

```
let count = 0;
const MAX = 100;    // block scoped, immutable binding
count += 1;
```

PHP

```
$count = 0;
const MAX = 100;    // compile-time constant (file/namespace scope)
define('RUNTIME_MAX', 100); // runtime-defined constant
$count += 1;
```

The biggest surprise: **PHP has no block scope and no closure-by-default over outer variables**. Variables are function-scoped, and a nested function or closure does *not* automatically see the enclosing scope — you must import variables explicitly with `use` (covered in 1.9). There is also no variable hoisting.

Concept	JavaScript	PHP
Mutable local	<code>let x = 1</code>	<code>\$x = 1</code>

Concept	JavaScript	PHP
Constant (module)	<code>const X = 1</code>	<code>const X = 1;</code>
Constant (runtime)	—	<code>define('X', 1)</code>
Class constant	<code>static readonly</code>	<code>const X = 1;</code> inside class
Scope unit	block <code>{ }</code>	function body
Undefined access	<code>undefined</code>	warning + <code>null</code>

COMMON MISTAKE

Expecting `if ($x = 5) { ... }` to be caught. PHP allows assignment in conditions just like JS, and with no block scope a variable "leaks" out of an `if`. Enable all warnings in development and lean on `===`.

1.2 Scalar Types: strings, numbers, booleans, null

PHP's scalar types are `int`, `float`, `string`, `bool`, plus `null`. There is no separate `BigInt` or `Symbol`; there is no distinction between number and integer at the syntax level, but `int` and `float` are genuinely different types at runtime (unlike JS where everything is an IEEE-754 `number`).

JAVASCRIPT / NODE.JS

```
const name = "Ahmed";
const greeting = `Hello, ${name}! You have ${3 + 4} messages.`;
const isActive = true;
const nothing = null;
```

PHP

```
$name = 'Ahmed'; // single quotes: no interpolation
$greeting = "Hello, {$name}! You have " . (3 + 4) . " messages.";
$isActive = true;
$nothing = null;
```

Two things jump out. First, **string interpolation only happens inside double quotes**, and the safe form uses braces: `"{$user->name}"`. Single quotes are literal (faster, and what you want for fixed strings). Second, **string concatenation uses `.`, not `+`**. In PHP `+` is strictly numeric; `"3" + "4"` is `7`, and `"a" + "b"` is a `TypeError` under strict types.

Operation	JavaScript	PHP
Concatenate	<code>a + b</code>	<code>a . b</code>
Interpolate	<code>`\${x}`</code>	<code>"\${x}"</code>
Template heredoc	<code>...multiline...</code>	<code><<<EOT ... EOT;</code>
Length	<code>s.length</code>	<code>strlen(\$s)</code> (bytes) / <code>mb_strlen(\$s)</code>

Operation	JavaScript	PHP
Contains	<code>s.includes(x)</code>	<code>str_contains(\$s, \$x)</code>
Split	<code>s.split(',')</code>	<code>explode(',', \$s)</code>
Join	<code>arr.join(',')</code>	<code>implode(',', \$arr)</code>
Format	template literal	<code>sprintf('%s is %d', \$n, \$a)</code>

NODE.JS MENTAL MODEL

PHP's standard library is a giant flat namespace of `str_*`, `array_*`, `mb_*` functions — the opposite of JavaScript's method-on-object style (`"x".toUpperCase()`). You call `strtoupper($s)`, not `$s.toUpperCase()`. It feels backwards for a day, then becomes muscle memory. For Unicode-safe work always prefer the `mb_*` (multibyte) variants.

1.3 Arrays and Associative Arrays

This is the concept that most reshapes how you think. **PHP has one array type that is simultaneously a JS array and a JS Map/object.** Integer keys give you a list; string keys give you a dictionary; you can mix both.

JAVASCRIPT / NODE.JS

```
const scores = [10, 20, 30];
const user = { name: "Ahmed", age: 31, roles: ["admin"] };
user.email = "a@x.dev";
const first = scores[0];
```

PHP

```
$scores = [10, 20, 30];           // list: keys 0,1,2
$user = [
    'name' => 'Ahmed',
    'age'  => 31,
    'roles' => ['admin'],
];
$user['email'] = 'a@x.dev';       // add a key
$first = $scores[0];
```

Arrays are **value types**, not references. Assigning an array copies it (copy-on-write under the hood). This is a profound difference from JS objects/arrays, which are always references.

JAVASCRIPT / NODE.JS

```
const a = [1, 2, 3];
const b = a;           // same reference
b.push(4);              // a is now [1,2,3,4]
```

PHP

```
$a = [1, 2, 3];
$b = $a;           // full copy (copy-on-write)
$b[] = 4;          // $a is STILL [1, 2, 3]
```

NODE.JS MENTAL MODEL

In JS you defensively `structuredClone()` or spread `[...a]` to avoid shared mutation. In PHP the *default* is the copy; you have to opt *into* sharing with a `&` reference (1.8). This flips your instincts: PHP array code is accidentally immutable, which is usually a feature.

Essential array functions (you will use these hourly):

Task	JavaScript	PHP
Append	<code>arr.push(x)</code>	<code>\$arr[] = \$x;</code>
Length	<code>arr.length</code>	<code>count(\$arr)</code>
Has key	<code>k in obj</code>	<code>array_key_exists(\$k, \$a) / isset(\$a[\$k])</code>
Keys	<code>Object.keys(o)</code>	<code>array_keys(\$a)</code>
Values	<code>Object.values(o)</code>	<code>array_values(\$a)</code>
Entries	<code>Object.entries(o)</code>	<code>foreach (\$a as \$k => \$v)</code>
Merge	<code>{...a, ...b}</code>	<code>[...\$a, ...\$b]</code> or <code>array_merge(\$a, \$b)</code>
Includes	<code>arr.includes(x)</code>	<code>in_array(\$x, \$arr, true)</code>
Find index	<code>arr.indexOf(x)</code>	<code>array_search(\$x, \$arr, true)</code>
Slice	<code>arr.slice(1,3)</code>	<code>array_slice(\$a, 1, 2)</code>

COMMON MISTAKE

`array_merge` rennumbers integer keys, so merging two lists concatenates them, but merging maps with the same string key lets the right side win — exactly like object spread. The spread operator `[...$a, ...$b]` behaves the same for lists but preserves string keys differently in edge cases; know which you want. For deep control, reach for explicit functions.

1.4 Comparison and Truthiness

PHP

```
0 == '0'           // true  (juggling) – avoid
0 === '0'          // false (strict)  – prefer
null ?? 'x'        // 'x'   (null coalescing, like JS ??)
$a <=> $b           // spaceship: -1, 0, or 1 (great for sorting)
```

PHP truthiness differs from JS in one memorable way: the string `"0"` is **falsy** in PHP, and so is the empty array `[]`. Everything else non-empty is truthy.

Value	JS truthy?	PHP truthy?
<code>""</code>	false	false
<code>"0"</code>	true	false
<code>"false"</code>	true	true
<code>0, 0.0</code>	false	false
<code>[]</code>	true	false
<code>null</code>	false	false

PRODUCTION TIP

Prefer explicit checks over truthiness at boundaries: `if ($s !== '')`, `if (count($items) > 0)`. It reads clearly and dodges the `"0"` trap that silently drops legitimate data.

1.5 Conditionals, Loops, and `match`

`if/else/while/for` are identical to JS. `foreach` is PHP's `for...of` and `for...in` combined, and it gives you the key *and* value in one form.

PHP

```
foreach ($users as $user) { echo $user['name']; }
foreach ($user as $key => $value) { echo "$key = $value"; }
for ($i = 0; $i < 10; $i++) { /* ... */ }
```

The modern star is `match` — an expression (returns a value) that uses **strict** `===` **comparison** and requires exhaustiveness (throws if nothing matches). It replaces the bug-prone `switch`.

JAVASCRIPT / NODE.JS

```
const label = (() => {
  switch (status) {
    case "active": return "On";
    case "paused": return "Hold";
    default: return "Unknown";
  }
})();
```

PHP

```
$label = match ($status) {
    'active'           => 'On',
    'paused', 'idle'   => 'Hold',
    default            => 'Unknown',
};
```

COMMON MISTAKE

Using `switch` and forgetting `break` — PHP `switch` falls through exactly like C/JS and compares with loose `==`. Prefer `match` for its strictness, value-return, and no-fallthrough behavior.

1.6 Functions, Defaults, Named & Variadic Arguments

JAVASCRIPT / NODE.JS

```
function makeUser(name, age = 18, ...roles) {
  return { name, age, roles };
}
makeUser("Ada", 30, "admin", "billing");
```

PHP

```
function makeUser(string $name, int $age = 18, string ...$roles): array
{
    return ['name' => $name, 'age' => $age, 'roles' => $roles];
}
makeUser('Ada', 30, 'admin', 'billing');
makeUser(name: 'Ada', age: 30); // named arguments
```

PHP has **named arguments** (like Python), which JS lacks — you can skip optional positional params by naming the one you want. Variadics use `...` on the parameter, and spreading a call uses `...` on the argument, exactly like JS.

Feature	JavaScript	PHP
Default param	<code>age = 18</code>	<code>int \$age = 18</code>
Rest/variadic	<code>...roles</code>	<code>string ...\$roles</code>
Spread call	<code>fn(...args)</code>	<code>fn(...\$args)</code>
Named args	✗ (use object)	<code>fn(age: 30)</code>
Return type	TS <code>: User</code>	<code>: User</code> (enforced at runtime)

1.7 Closures, Arrow Functions, and Callbacks

Here is the sharpest divergence for a JS developer. **PHP closures do not capture the surrounding scope automatically.** A long closure (`function () {}`) must list captured variables via `use`. A short arrow (`fn () =>`) *does* auto-capture by value, but only allows a single expression.

JAVASCRIPT / NODE.JS

```
const tax = 0.15;
const withTax = (p) => p * (1 + tax);           // captures tax automatically
const totals = prices.map((p) => withTax(p));
```

PHP

```
$tax = 0.15;
$withTax = fn($p) => $p * (1 + $tax);           // arrow auto-captures by value
$long = function ($p) use ($tax) {             // long closure needs `use`
    return $p * (1 + $tax);
};
$totals = array_map($withTax, $prices);
```

Capture is **by value** by default; add `&` to capture by reference: `function () use (&$total) { $total += 1; }`. Closures are first-class values you can store, pass, and return — same as JS functions.

NODE.JS MENTAL MODEL

Think of the long-form `use (...)` as writing out, by hand, exactly which variables the closure "closes over." JS does this implicitly; PHP makes it explicit, which prevents accidental capture of huge scopes but costs a few keystrokes. Arrow `fn()` is the ergonomic middle ground for one-liners.

1.8 References

A `&` makes a true alias — two names for the same slot. Use sparingly; they are the exception, not the rule, and they fight PHP's copy-by-default model.

PHP

```
$a = 1;
$b = &$a;    // $b aliases $a
$b = 5;      // $a is now 5
function addTax(array &$order): void { $order['total'] *= 1.15; } // mutates caller
```

WHEN NOT TO USE REFERENCES

Almost always. If you find yourself reaching for `&` to "return multiple values" or "avoid copying," prefer returning a new array/DTO. References make code hard to reason about and are a frequent source of spooky bugs. Copy-on-write already makes value passing cheap.

1.9 Objects, Classes, and Constructor Promotion

PHP classes are conventional class-based OOP (much closer to Java/C# than to JS's prototype model). Modern PHP has **constructor property promotion**, which collapses the boilerplate you know from TypeScript.

TYPESCRIPT

```
class Money {
  constructor(
    public readonly amount: number,
    public readonly currency: string = "USD",
  ) {}
  format(): string { return `${this.amount} ${this.currency}`; }
}
```

PHP

```
final class Money
{
    public function __construct(
        public readonly int $amount,
        public readonly string $currency = 'USD',
    ) {}

    public function format(): string
    {
        return "{$this->amount} {$this->currency}";
    }
}

$m = new Money(100, 'EUR');
echo $m->format(); // property/method access uses ->, not .
```

Member access is `->` (instance) and `::` (static/class-level), never `.` (that is string concat). `$this` is the instance. `readonly` properties can be set once (in the constructor) and never again — real immutability, stronger than TS's compile-only `readonly`.

Concept	TypeScript	PHP
Instance access	<code>this.name</code>	<code>\$this->name</code>
Static access	<code>Cls.make()</code>	<code>Cls::make()</code>
Constant access	<code>Cls.X</code>	<code>Cls::X</code>
New instance	<code>new Cls()</code>	<code>new Cls()</code>
Immutable field	<code>readonly x</code>	<code>public readonly int \$x</code>
Nullable field	<code>x?: string</code>	<code>?string \$x</code>

1.10 Interfaces, Abstract Classes, Traits, Enums

Interfaces and abstract classes work as you expect from TypeScript/Java. The PHP-specific tool is the **trait** — a reusable block of methods mixed into a class (horizontal reuse, like a mixin), solving the "no multiple inheritance" problem.

PHP


```

interface Notifiable { public function routeFor(string $channel): string; }

abstract class Model {
    abstract public function table(): string;
    public function save(): void { /* shared logic */ }
}

trait HasTimestamps {                // mixin of methods + state
    public ?string $createdAt = null;
    public function touch(): void { $this->createdAt = date('c'); }
}

final class User extends Model implements Notifiable {
    use HasTimestamps;                // mix the trait in
    public function table(): string { return 'users'; }
    public function routeFor(string $channel): string { return $this->email; }
}

```

Enums (PHP 8.1+) are a highlight for TS developers — they are real types, can be *backed* by a string/int, and can carry methods.

TYPESCRIPT

```
enum Status { Active = "active", Paused = "paused" }
```

PHP

```

enum Status: string {
    case Active = 'active';
    case Paused = 'paused';

    public function label(): string {
        return match ($this) {
            Status::Active => 'Active',
            Status::Paused => 'On hold',
        };
    }
}

Status::from('active');           // -> Status::Active (throws if invalid)
Status::tryFrom('nope');         // -> null
Status::Active->value;            // 'active'

```

PRODUCTION TIP

Back your enums with strings and use them everywhere a status/role/type lives — as Eloquent casts (Part 7), match arms, and validation rules. They give you exhaustiveness checking and self-documenting domains that stringly -typed JS constants never provide.

1.11 Type Declarations: union, nullable, intersection

Modern PHP types are enforced **at runtime**, not erased like TypeScript.

PHP

```
function find(int|string $id): ?User { /* ... */ } // union + nullable
function log(Stringable&JsonSerializable $x): void {} // intersection
function first(iterable $items): mixed {} // iterable, mixed
```

Type feature	TypeScript	PHP
Union	<code>number string</code>	<code>int string</code>
Nullable	<code>string null / x?</code>	<code>?string</code>
Intersection	<code>A & B</code>	<code>A&B</code>
Any	<code>any / unknown</code>	<code>mixed</code>
Void	<code>void</code>	<code>void</code>
Never	<code>never</code>	<code>never</code>
Enforcement	compile-time only	runtime

NODE.JS MENTAL MODEL

TypeScript types vanish at build time; a wrong type only bites if it reaches JS logic. PHP types are checked by the engine on every call, so a type violation is a hard runtime `TypeError` at the boundary. You get less expressive generics than TS, but stronger runtime guarantees for free.

1.12 Namespaces, Autoloading, Exceptions, Generators, Attributes

Namespaces replace ES modules for organizing code; **Composer autoloading** (Part 4) replaces `import`. You declare a namespace, then `use` classes.

PHP

```
namespace App\Services;

use App\Models\User;
use RuntimeException;

final class Billing {
    public function charge(User $user, int $cents): void {
        if ($cents <= 0) {
            throw new RuntimeException('amount must be positive');
        }
    }
}
```

Exceptions work like JS `try/catch/finally`, but you catch by *type*, which is far more precise than JS's single `catch (e)`.

PHP

```
try {
    $billing->charge($user, $amount);
} catch (PaymentDeclined $e) {
    // handle a specific domain error
} catch (RuntimeException $e) {
    report($e);
} finally {
    $lock->release();
}
```

Generators (`yield`) give you lazy iteration, exactly like JS `function*` — crucial for streaming large datasets without loading them all into memory.

PHP

```
function readLines(string $path): Generator {
    $fh = fopen($path, 'r');
    while (($line = fgets($fh)) !== false) {
        yield trim($line);           // one line at a time, O(1) memory
    }
    fclose($fh);
}
foreach (readLines('big.csv') as $line) { /* ... */ }
```

Attributes (`#[...]`) are PHP's decorators — structured metadata read via Reflection. Laravel and PHP frameworks use them for routing, validation, and DI. They are declarative annotations, not behavior wrappers like some JS decorators.

PHP

```
#[Route('/users/{id}', methods: ['GET'])]
public function show(int $id): Response { /* ... */ }
```

EXERCISE 1.2

Port a small Node module to PHP: define a `Money` value object with `readonly` fields and an `add()` method that returns a *new* `Money`; a `Currency` backed enum; and a generator `range($start, $end)` that yields integers lazily. Add `declare(strict_types=1)` and confirm that passing a `float` where `int` is expected throws a `TypeError`.

Part 2 · Functional Programming in PHP

You already think functionally in JavaScript: `filter().map().reduce()`, pure functions, composition, immutability by convention. PHP supports every one of these ideas, but with two twists that matter. First, the array functions are **free functions with an awkward argument order** (`array_map`, `array_filter`, `array_reduce`) rather than chainable methods. Second, Laravel ships **Collections**, which restore the fluent, chainable pipeline you love — and go far beyond what `Array.prototype` offers.

2.1 First-Class Functions and Higher-Order Functions

Functions are values. You can pass them, return them, and store them. You reference a named function as a string or a first-class callable with `(...)`.

JAVASCRIPT / NODE.JS

```
const double = (x) => x * 2;
const apply = (fn, x) => fn(x);
apply(double, 21);           // 42
[1, 2, 3].map(double);
```

PHP

```
$double = fn($x) => $x * 2;
$apply = fn(callable $fn, $x) => $fn($x);
$apply($double, 21);           // 42
array_map($double, [1, 2, 3]);
array_map('strtoupper', $words); // named function as a string
array_map(strtoupper(...), $words); // first-class callable syntax (8.1+)
```

The `callable` type hint documents "this parameter is a function." The `strtoupper(...)` syntax turns any named function or method into a closure — the modern, IDE-friendly way to pass built-ins.

2.2 map / filter / reduce — the argument-order trap

The functions do the same jobs as their JS counterparts, but the **argument order is inconsistent**, which is the single biggest day-one annoyance.

JAVASCRIPT / NODE.JS

```
const activeNames = users
  .filter((u) => u.active)
  .map((u) => u.name)
```

```
.sort());
```

```
const total = items.reduce((sum, i) => sum + i.price, 0);
```

PHP

```
// array_map:    callback FIRST, then array(s)
$names = array_map(fn($u) => $u['name'], $users);

// array_filter: array FIRST, then callback (opposite order!)
$active = array_filter($users, fn($u) => $u['active']);

// array_reduce: array, callback, initial
$total = array_reduce($items, fn($sum, $i) => $sum + $i['price'], 0);
```

Operation	JavaScript	PHP (raw)	Argument order
map	<code>arr.map(fn)</code>	<code>array_map(fn, arr)</code>	fn, arr
filter	<code>arr.filter(fn)</code>	<code>array_filter(arr, fn)</code>	arr, fn
reduce	<code>arr.reduce(fn, i)</code>	<code>array_reduce(arr, fn, i)</code>	arr, fn, init
some	<code>arr.some(fn)</code>	<i>(no builtin — use loop / Collection)</i>	—
every	<code>arr.every(fn)</code>	<i>(no builtin — use loop / Collection)</i>	—
sort	<code>arr.sort(cmp)</code>	<code>usort(\$arr, cmp)</code> (mutates!)	arr(ref), cmp
flat	<code>arr.flat()</code>	<code>array_merge(...\$arr)</code>	—

COMMON MISTAKE

Three traps bite Node developers here. (1) `array_map` / `array_filter` disagree on argument order — memorize it or use Collections. (2) `array_filter` **preserves original keys**, so a filtered list becomes a map with gaps (`[0 => .., 2 => ..]`); wrap with `array_values()` to re-index. (3) `usort` **mutates in place** and returns `bool`, not the sorted array — a classic "why is my variable `true`?" bug.

2.3 Why raw PHP FP gets ugly — and the fix

Chaining `array_*` calls reads inside-out, the opposite of a fluent pipeline:

PHP

```
// reads backwards: reduce(map(filter(...))) — hard to follow
$total = array_reduce(
  array_map(
    fn($o) => $o['price'] * $o['qty'],
    array_filter($orders, fn($o) => $o['paid'])
  ),
  fn($sum, $line) => $sum + $line,
```

```
0
);
```

This is where **Laravel Collections** change everything. `collect()` wraps an array in an object with 100+ chainable, immutable-by-default methods whose names and semantics mirror `Array.prototype` — often more closely than raw PHP does.

PHP

```
use Illuminate\Support\Collection;

$total = collect($orders)
    ->filter(fn($o) => $o['paid'])
    ->map(fn($o) => $o['price'] * $o['qty'])
    ->sum();
```

JAVASCRIPT / NODE.JS

```
const total = orders
  .filter((o) => o.paid)
  .map((o) => o.price * o.qty)
  .reduce((s, x) => s + x, 0);
```

That is a one-to-one mental mapping. For anything beyond a single `array_map`, prefer Collections.

2.4 The Collection pipeline vocabulary

Collections are the functional heart of Laravel. Every Eloquent query returns one, so this vocabulary pays off constantly.

You want	JS Array	Laravel Collection
Transform	<code>map</code>	<code>map</code>
Keep some	<code>filter</code>	<code>filter</code> / <code>reject</code>
Fold	<code>reduce</code>	<code>reduce</code>
Any/all	<code>some</code> / <code>every</code>	<code>contains</code> / <code>every</code>
Flatten-map	<code>flatMap</code>	<code>flatMap</code>
Group by key	<i>(manual)</i>	<code>groupBy</code>
Index by key	<i>(manual <code>Map</code>)</i>	<code>keyBy</code>
Pull one field	<code>map(x=>x.f)</code>	<code>pluck('f')</code>
Unique	<code>[...new Set()]</code>	<code>unique</code>
Sort by field	<code>sort((a,b)=>...)</code>	<code>sortBy('field')</code>
Take / skip	<code>slice</code>	<code>take</code> / <code>skip</code>
Side effect	<code>forEach</code>	<code>each</code>

You want	JS <code>Array</code>	Laravel <code>Collection</code>
Chunk	<i>(manual)</i>	<code>chunk(100)</code>
First match	<code>find</code>	<code>first(fn)</code>

PHP

```
$report = collect($orders)
  ->groupBy('customer_id') // -> Collection of Collections
  ->map(fn($group) => [
    'orders' => $group->count(),
    'revenue' => $group->sum('total'),
  ])
  ->sortByDesc('revenue')
  ->take(10);
```

`pipe`, `pipeThrough`, `tap`, `when`, `unless` give you flow control inside a chain — the pieces raw JS lacks:

PHP

```
collect($users)
  ->when($onlyActive, fn($c) => $c->where('active', true)) // conditional step
  ->tap(fn($c) => logger()->info('count', [$c->count()])) // peek, no change
  ->map(fn($u) => $u['email'])
  ->pipe(fn($c) => $c->implode(', ')); // hand the whole
                                     // collection to a fn
```

- `tap($fn)` runs a side effect and returns the **same** collection (like a debugging `console.log` that doesn't break the chain).
- `when($cond, $fn)` applies `$fn` only if `$cond` is truthy — no more breaking a fluent chain to insert an `if`.
- `pipe($fn)` passes the entire collection to `$fn` and returns its result.

NODE.JS MENTAL MODEL

Collections are what you would get if you extended `Array.prototype` with `lodash` and `RxJS`-style operators, made it immutable by default, and gave it first-class `when / tap`. When you see `collect(...)->...`, read it as a `lodash` `_.chain()` that everyone on the team actually agrees to use.

2.5 Currying, Partial Application, Composition, Pipe

PHP has no built-in `compose / pipe / curry`, but they are trivial to write because closures are first-class.

JAVASCRIPT / NODE.JS

```
const pipe = (...fns) => (x) => fns.reduce((v, f) => f(v), x);
const clean = pipe(trim, toLower, slugify);
```

```
const addTax = (rate) => (price) => price * (1 + rate); // curried
const withVat = addTax(0.15);
```

PHP

```
function pipe(callable ...$fns): callable {
    return fn($x) => array_reduce($fns, fn($v, $f) => $f($v), $x);
}
$clean = pipe(trim(...), strtolower(...), slugify(...));

$addTax = fn(float $rate) => fn(float $price) => $price * (1 + $rate); // curried
$withVat = $addTax(0.15);
$withVat(100.0); // 115.0
```

WHEN NOT TO OVER-FUNCTIONAL-IZE

You *can* build a point-free, curried, composed cathedral in PHP. You usually **should not**. The idiomatic Laravel codebase uses Collections for data pipelines and plain methods for logic. A hand-rolled `curry()` helper that saves three characters but puzzles the next developer is a net loss. Reach for composition when it genuinely removes duplication, not to prove it can be done.

2.6 Lazy Evaluation: Generators and LazyCollection

Eager Collections load everything into memory. For large or infinite streams, use **LazyCollection**, which is backed by generators and evaluates one item at a time — the equivalent of a Node stream or an async iterator.

PHP

```
use Illuminate\Support\LazyCollection;

LazyCollection::make(function () {
    $fh = fopen('huge.csv', 'r');
    while (($row = fgets($fh)) !== false) {
        yield str_getcsv($row); // yields lazily, 0(1) memory
    }
})
->filter(fn($row) => $row[3] === 'paid')
->take(1000) // stops reading the file after 1000
->each(fn($row) => Import::dispatch($row));
```

JAVASCRIPT / NODE.JS

```
async function* readCsv(path) { /* yield rows */ }
// consume lazily with for await ... of
```


PRODUCTION TIP

Eloquent's `->cursor()` returns a `LazyCollection` that streams rows from the database one at a time instead of hydrating a million models at once — the antidote to "the report job ran out of memory." Reach for it whenever a result set could be large. `->chunk(500)` is the batched cousin for writes.

EXERCISE 2.1

Given `$orders` (each with `customer`, `total`, `paid`), produce with a single `Collection` pipeline: the top 5 customers by paid revenue, as `['name' => ..., 'revenue' => ...]`, sorted descending. Then rewrite the same logic with raw `array_filter` / `array_map` / `array_reduce` and compare readability. Finally, convert a 1-million-row CSV summation to a `LazyCollection` and confirm memory stays flat.

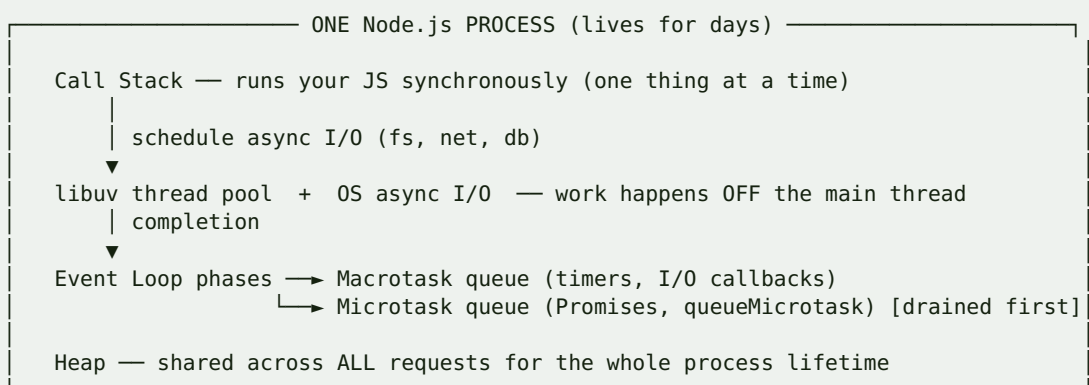
Part 3 · The Node Event Loop vs the PHP Runtime

This is the most important conceptual chapter in the book. If Part 1 taught you PHP's *syntax*, this teaches you its *physics*. Almost every architectural decision in a Laravel app — queues, workers, Octane, why global caching behaves differently — flows from one fact: **Node and classic PHP have opposite execution models**. Get this chapter into your bones and the rest of the book clicks into place.

3.1 The Node.js model, restated precisely

A Node process is a single long-lived program with one main thread running an **event loop** on top of **libuv**. It boots once and then lives for days, juggling thousands of concurrent connections by never blocking: I/O is handed off to the OS/libuv, and your callbacks run when results are ready.

DIAGRAM



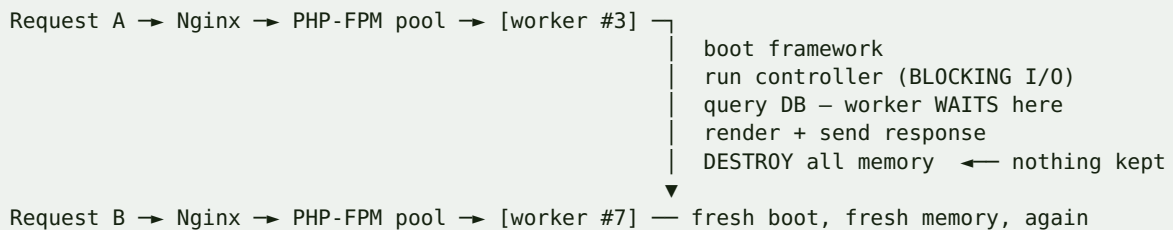
Key consequences you rely on every day:

- **Concurrency without threads.** One thread handles many in-flight requests because nothing blocks while waiting on I/O.
- **Shared memory across requests.** A module-level `Map` cache, a connection pool, a warmed config — all persist between requests for free.
- `await` **yields the loop.** While one request awaits the database, the loop serves others.
- **CPU-bound work is dangerous.** A synchronous 200 ms loop freezes *every* connection; you offload to Worker Threads or child processes.

3.2 The classic PHP model: shared-nothing, one request per process

Classic PHP (PHP-FPM behind Nginx) is the mirror image. There is **no application-level event loop and no shared memory between requests**. Each request is served by a worker process that boots your framework, runs your code top to bottom **synchronously**, sends the response, and then **throws everything away**.

DIAGRAM



The PHP-FPM pool is a set of OS processes (say 20). Concurrency comes from **having many worker processes**, not from one process multiplexing. If all 20 workers are busy waiting on slow queries, request 21 waits in Nginx's queue.

Dimension	Node.js	Classic PHP (FPM)
Process lifetime	boots once, lives for days	one request, then discarded
Concurrency model	event loop, 1 thread, non-blocking	N OS worker processes
I/O	non-blocking, <code>await</code> yields	blocking ; worker waits
Memory between requests	shared (heap persists)	shared-nothing (fresh)
Global mutable state	persists (careful: leaks)	safe; wiped each request
CPU-bound task	blocks the whole loop	blocks only that one worker
Bytecode	JIT-compiled, warm	recompiled per request unless OPcache
"Restart to apply code"	yes (process holds old code)	no; next request sees new code

NODE.JS MENTAL MODEL

Node is a restaurant with **one hyper-efficient waiter** who never stands idle — while one table's food cooks, the waiter serves others. Classic PHP is a restaurant with **twenty waiters, each of whom serves exactly one table then goes home**, and a new waiter is hired for the next table. Neither is "better"; they fail and scale differently, and you tune them differently.

3.3 Why this is often *simpler*, not just different

The shared-nothing model has a genuine upside that Node developers underappreciate: **most concurrency bugs cannot exist**. There is no shared mutable heap across requests, so no data races between requests, no "I leaked a per-user value into a module singleton,"

no cache poisoning from request A bleeding into request B. Each request is an isolated universe. You write plain top-to-bottom code and the runtime guarantees isolation.

The downside is the mirror image: **you cannot keep anything warm in process memory**. No in-process cache surviving requests, no persistent WebSocket connections held in a `Map`, no background timer ticking between requests. Those capabilities move *out* of the request into dedicated infrastructure — which is exactly what Parts 13–15 are about.

COMMON MISTAKE

Porting a Node pattern like "cache the config in a module-level variable on first request, reuse forever." In classic PHP that variable is gone the instant the response is sent; you'd recompute it every request. The PHP answer is an *external* cache (Redis/APCu) shared across workers — Part 15.

3.4 OPcache and the bootstrap cost

Because PHP recompiles source to bytecode on each request, PHP without OPcache would be absurdly slow. **OPcache** caches compiled bytecode in shared memory across all FPM workers, so the *compile* step is paid once, not per request. Your framework still **re-executes** bootstrap logic each request (building the container, registering providers), which is the cost Octane later eliminates.

INI / .ENV

```
; php.ini – always on in production
opcache.enable=1
opcache.memory_consumption=256
opcache.max_accelerated_files=20000
opcache.validate_timestamps=0 ; prod: don't stat files; clear cache on deploy
```

PRODUCTION TIP

In production set `opcache.validate_timestamps=0` so PHP never checks the filesystem for changes — then run `php artisan optimize` / restart FPM on deploy to load new code. This is the closest analog to "the Node process holds old code until restart": here *OPcache* holds old bytecode until you clear it.

3.5 Blocking I/O and where concurrency comes from

In Node you fire three independent HTTP calls and `await Promise.all` them; they overlap. In a **single** classic PHP request, the default HTTP/DB calls **block** — three sequential API calls take the sum of their latencies, because the worker literally waits on each socket. Concurrency across *different* requests still happens (other workers serve them), but concurrency *within* one request needs explicit tools.

JAVASCRIPT / NODE.JS

```
// Node: these three run concurrently
const [a, b, c] = await Promise.all([
  fetch(urlA), fetch(urlB), fetch(urlC),
]);
```

PHP

```
// Laravel: concurrent HTTP within one request using pooled requests
use Illuminate\Support\Facades\Http;

$responses = Http::pool(fn ($pool) => [
  $pool->get($urlA),
  $pool->get($urlB),
  $pool->get($urlC),
]); // fired concurrently via curl_multi under the hood
```

PHP

```
// Laravel Concurrency facade (11+): run callbacks in parallel processes
use Illuminate\Support\Facades\Concurrency;

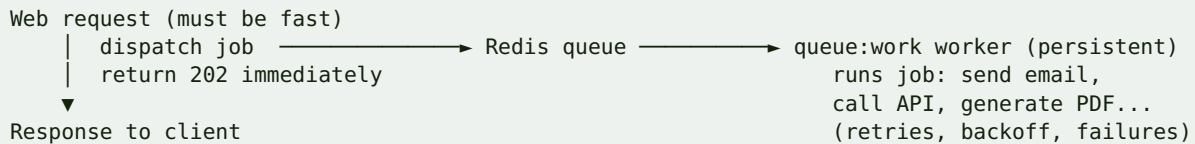
[$users, $orders] = Concurrency::run([
  fn () => User::count(),
  fn () => Order::sum('total'),
]);
```

Node concurrency tool	PHP / Laravel counterpart
<code>Promise.all(...)</code>	<code>Http::pool(...)</code> for HTTP; <code>Concurrency::run(...)</code>
<code>await fetch()</code>	blocking <code>Http::get()</code> (sequential by default)
Worker Threads (CPU)	queue jobs on workers; <code>Concurrency</code> (separate procs)
<code>child_process</code>	<code>Process::run()</code> / <code>Process::pool()</code> (Part 19)
background timer	Scheduler + cron (Part 14)
in-process job queue	Redis-backed Queue + worker processes (Part 13)
persistent WS server	Reverb / broadcasting via a separate server (Part 21)

3.6 Moving work off the request: queues, workers, scheduler

Since a PHP request must finish quickly and cannot spawn background timers, slow or deferred work moves onto **queues** processed by **long-running worker processes** (`php artisan queue:work`). Those workers *are* persistent PHP processes — the one place classic PHP keeps memory warm — so they behave more like a small Node service.

DIAGRAM



This is why a Laravel app in production is not one process but a *fleet*: Nginx + FPM workers for requests, `queue:work` workers for background jobs, a cron entry driving the scheduler, and optionally Horizon to manage it all (Part 20).

NODE.JS MENTAL MODEL

In Node you might do background work with `setImmediate`, an in-process BullMQ worker in the same process, or a detached async function. In PHP that work is almost always a **queued job on a separate worker process**, because the web request has no "later" — it ends. BullMQ's mental model maps almost exactly onto Laravel Queues (Part 13); you already know the shape.

3.7 Laravel Octane: giving PHP a Node-like runtime

Octane boots your Laravel application **once** into a long-lived worker (via Swoole, or RoadRunner, or FrankenPHP) and keeps it in memory, serving many requests from the warmed app — eliminating the per-request bootstrap and delivering large throughput gains. In other words, Octane makes PHP behave much more like Node: one process, warm memory, many requests.

That power reintroduces Node-style hazards you were previously immune to:

PHP

```
// DANGER under Octane: static state now persists ACROSS requests
class Cart {
    public static array $items = []; // leaks between users under Octane!
}
```

Concern	Classic FPM	Octane
App bootstrap	every request	once , then reused
Memory between requests	wiped	persists (Node-like)
Global/static state leak risk	none	real — must reset
Throughput	good	much higher
Mental model	shared-nothing	shared, like Node

COMMON MISTAKE

Adopting Octane and then storing per-request data in static properties, singletons, or the container without resetting. Under FPM this was harmless; under Octane it leaks one user's data into the next request — precisely the class of bug Node developers know as "module-level state." Octane gives you Node's speed *and* Node's footguns.

WHEN NOT TO REACH FOR OCTANE

Do not start a new project on Octane to "make it fast" before you have a reason. Classic FPM is simpler, safer, and fast enough for the vast majority of apps, and it forgives the shared-nothing mistakes above. Adopt Octane when profiling shows bootstrap/throughput is the real bottleneck and the team understands the state-reset discipline.

EXERCISE 3.1

(1) Draw the lifecycle of one of your Express endpoints and its Laravel-FPM equivalent, labeling where blocking happens. (2) Take an endpoint that makes three sequential external API calls and rewrite it with `Http::pool`; measure the latency difference. (3) In two sentences, explain why a module-level cache that works in Express silently fails under classic PHP but *would* work under Octane — and why that is dangerous.

Part 4 · Composer — PHP's npm

Composer is to PHP what npm is to Node: dependency resolution, a lockfile, autoloading, and scripts. The concepts map almost one-to-one, so this is a short chapter — mostly a translation table plus the few places the models diverge.

4.1 The direct translation

npm / Node	Composer / PHP	Notes
<code>package.json</code>	<code>composer.json</code>	manifest
<code>package-lock.json</code>	<code>composer.lock</code>	exact resolved versions — commit it
<code>node_modules/</code>	<code>vendor/</code>	installed code — gitignore it
<code>npm install</code>	<code>composer install</code>	install from lockfile
<code>npm install pkg</code>	<code>composer require pkg</code>	add a dependency
<code>npm install -D pkg</code>	<code>composer require --dev pkg</code>	dev dependency
<code>npm uninstall pkg</code>	<code>composer remove pkg</code>	
<code>npm update</code>	<code>composer update</code>	re-resolve within constraints
<code>npm run x</code>	<code>composer run x</code>	script
registry: npmjs.com	Packagist.org	public package index
scoped <code>@org/pkg</code>	<code>vendor/package</code>	every package is <code>vendor/name</code>

SHELL

```
composer create-project laravel/laravel myapp # like `npm create-...`
composer require laravel/sanctum             # add runtime dep
composer require --dev pestphp/pest          # add dev dep
composer install --no-dev --optimize-autoloader # production install
composer dump-autoload -o                     # rebuild autoload map
```

4.2 Version constraints and semantic versioning

Both ecosystems use semver, but the default caret differs subtly. npm's `^1.2.3` allows `<2.0.0`. Composer's `^1.2.3` means the same for `1.x`, but Composer also has the `~` "tilde" with a narrower meaning and the compound syntax.

Constraint	Meaning
<code>^1.2.3</code>	<code>>=1.2.3 <2.0.0</code> (allow minor+patch) — the default

Constraint	Meaning
<code>~1.2.3</code>	<code>>=1.2.3 <1.3.0</code> (allow patch only)
<code>~1.2</code>	<code>>=1.2.0 <2.0.0</code>
<code>1.2.*</code>	any patch of 1.2
<code>>=1.2 <1.5</code>	explicit range

PRODUCTION TIP

Commit `composer.lock` and deploy with `composer install` (never `composer update` on the server) — this is the exact same discipline as `npm ci` against `package-lock.json`. `composer update` is a *developer* action that re-resolves and rewrites the lock; run it locally, review the diff, test, commit.

4.3 Autoloading and PSR-4 — the part with no npm analog

Here is the one genuinely new idea. Node resolves modules at runtime via `require/import` walking `node_modules`. PHP instead builds a **class autoloader**: a map from *namespace* to *directory*, following the **PSR-4** standard. When you reference `App\Services\Billing`, Composer's autoloader knows to load `app/Services/Billing.php`. You almost never write `require` manually.

JSON

```
{
  "autoload": {
    "psr-4": {
      "App\\": "app/",
      "Database\\Factories\\": "database/factories/"
    },
    "files": ["app/helpers.php"]
  },
  "require": {
    "php": "^8.3",
    "laravel/framework": "^12.0"
  },
  "require-dev": {
    "pestphp/pest": "^3.0"
  },
  "scripts": {
    "test": "pest",
    "post-autoload-dump": ["@php artisan package:discover"]
  }
}
```

The rule: **namespace prefix** ↔ **base directory**, then **namespace segments** ↔ **subfolders**, **class name** ↔ **filename**. So `App\Http\Controllers\UserController` must live at

`app/Http/Controllers/UserController.php`. Get the file path or the namespace wrong and you get "class not found."

PHP

```
// app/Services/Billing.php
namespace App\Services;      // MUST match the PSR-4 mapping "App\" => "app/"

final class Billing { /* ... */ }
```

PHP

```
// somewhere else – no require needed, the autoloader finds it
use App\Services\Billing;
$b = new Billing();
```

NODE.JS MENTAL MODEL

Instead of `import { Billing } from '../..services/billing.js'` with a real filesystem path, PHP uses a *logical* name (`App\Services\Billing`) that the autoloader resolves to a file via the PSR-4 convention. It is closer to TypeScript path aliases (`@/services/billing`) than to relative imports — but it is enforced by a naming convention, not a bundler config.

COMMON MISTAKE

Renaming a class or moving a file and forgetting the namespace/path must stay in sync, then seeing "Class not found." Also: after adding a `files-autoloaded` helper or changing mappings, run `composer dump-autoload` — the autoload map is generated, not live. In production always use `--optimize-autoloader` (`-o`) to build a fast classmap.

4.4 Scripts and package development

`composer.json` `scripts` mirror npm scripts and can hook lifecycle events (`post-install-cmd`, `post-autoload-dump`). To publish a package, you tag a Git release and submit the repo to Packagist — there is no separate "publish" upload step like `npm publish`; Packagist reads your Git tags.

SHELL

```
composer run test           # run the "test" script
composer require myvendor/mypkg # consumers install by vendor/name
```

EXERCISE 4.1

Create `composer.json` for a small library namespaced `Acme\Toolkit\` mapped to `src/`. Add one class, autoload it, and require it from a test script. Add a dev dependency on Pest and a `test` script. Explain, in one sentence each, the roles of `composer.lock`, `vendor/`, and `dump-autoload`.

Part 5 · Laravel Architecture for Express Developers

In Express you assemble an application by hand: you `require` things, `new` your dependencies, and wire middleware in the order you want. Laravel inverts this. It boots a **service container** that constructs and injects your dependencies, runs your request through a **middleware pipeline**, and resolves everything by **convention**. If Express is a pile of Lego bricks, Laravel is a pre-designed kit — you can still build anything, but the framework holds the structure.

The closest Node analog to Laravel's architecture is **NestJS**: dependency injection, providers, a module/container graph, decorators/attributes, guards/interceptors. If you know Nest, you know 70% of Laravel's shape already.

5.1 Project structure

DIAGRAM

```

app/
  Http/
    Controllers/  - request handlers (like Express route handlers)
    Middleware/   - pipeline stages (like Express middleware)
    Requests/     - Form Request validation classes (like Zod schemas)
    Resources/    - response transformers (API serializers)
  Models/        - Eloquent models (like Prisma models + repo methods)
  Providers/      - service providers (bootstrap/wiring)
  Jobs/ Events/ Listeners/ Notifications/ Policies/ - domain building blocks
bootstrap/app.php - application assembly (routes, middleware, exceptions)
config/          - config files reading from env
database/
  migrations/ seeders/ factories/
routes/
  web.php api.php - route definitions
resources/views/  - Blade templates
tests/           - Pest/PHPUnit tests
  
```

NODE.JS MENTAL MODEL

Express gives you an empty `index.js` and total freedom (and total responsibility). Laravel gives you a filled-in cabinet where every kind of thing has a drawer. The learning curve is *learning the drawers*; the payoff is that every Laravel codebase looks the same, so you can navigate a stranger's project instantly.

5.2 The request lifecycle

Every request flows through the same pipeline. Compare it to an Express app's `app.use()` chain ending in a route handler:

DIAGRAM

```

Public endpoint (public/index.php)
  |
  ▼
bootstrap/app.php — build the Application (the service container)
  |
  ▼
Global middleware — e.g. maintenance mode, trim strings, CORS
  |
  ▼
Router matches route — applies route/group middleware (auth, throttle)
  |
  ▼
Controller / route action — your code runs, deps injected from container
  |
  ▼
Response travels BACK OUT through middleware (can modify response)
  |
  ▼
Send response

```

The crucial idea: middleware is a **pipeline** (an onion), just like Express. Code before `$next($request)` runs on the way in; code after it runs on the way out. This is the same "each layer wraps the next" model as Express's `(req, res, next)`.

5.3 The Service Container and Dependency Injection

The **service container** is the heart of Laravel. It is an object that knows how to build other objects and their dependencies — an IoC (Inversion of Control) container. When Laravel needs a controller, it inspects the constructor's type-hints and **automatically resolves and injects** each dependency. You rarely call `new` for services.

JAVASCRIPT / NODE.JS

```

// Express: you wire dependencies by hand
const mailer = new Mailer(config.smtp);
const billing = new Billing(mailer, db);
app.post("/charge", (req, res) => billing.charge(/* ... */));

```

PHP

```

// Laravel: the container injects dependencies by type-hint
final class ChargeController
{
    public function __construct(
        private readonly Billing $billing,    // auto-resolved & injected
    ) {}

    public function store(ChargeRequest $request): JsonResponse
    {
        $this->billing->charge($request->user(), $request->integer('cents'));
        return response()->json(status: 202);
    }
}

```

You never instantiated `Billing` or its own dependencies — the container walked the constructor graph and built them. This is **autowiring**, exactly like NestJS providers.

Binding: teaching the container how to build things

For interfaces (which cannot be `new` ed), or for objects needing configuration, you register a **binding** in a service provider.

PHP

```
// A contract (interface) and its implementation
interface PaymentGateway { public function charge(int $cents): string; }
final class StripeGateway implements PaymentGateway { /* ... */ }

// In a service provider's register() method:
$this->app->bind(PaymentGateway::class, StripeGateway::class);           // new each
time
$this->app->singleton(PaymentGateway::class, fn () => new StripeGateway( // one
shared
    config('services.stripe.secret')
));
```

Now any class type-hinting `PaymentGateway` receives a `StripeGateway`. Swap the binding and the whole app switches implementations — the textbook benefit of dependency injection and the key to testable code.

Binding	Meaning	Node analog
<code>bind(A, B)</code>	resolve <code>A</code> to a new <code>B</code> each time	factory function
<code>singleton(A, B)</code>	resolve <code>A</code> to one shared <code>B</code>	module-level singleton
<code>instance(A, \$obj)</code>	register an existing object	pre-built export
<code>bind</code> + closure	custom construction logic	factory with config
contextual binding	different impl for different consumers	manual DI wiring

NODE.JS MENTAL MODEL

In Express you build a "manual DI" file where you `new` everything in the right order and pass it around. The service container automates that wiring by reading type-hints, so adding a dependency is just adding a constructor parameter — no central wiring file to edit. It is NestJS's `@Injectable` providers without the decorators.

5.4 Service Providers — the bootstrap layer

Service providers are where you register bindings and run bootstrap logic. They have two methods: `register()` (bind things into the container — do not use other services here yet) and `boot()` (run after everything is registered — safe to use other services). This is Laravel's equivalent of the setup code at the top of your Express `index.js`, but organized into cohesive units.

PHP

```
final class BillingServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        $this->app->singleton(PaymentGateway::class,
            fn () => new StripeGateway(config('services.stripe.secret')));
    }

    public function boot(): void
    {
        // e.g. register a validation rule, a route macro, an event listener
    }
}
```

5.5 Facades and Contracts — the "magic" demystified

You will see static-looking calls everywhere: `Cache::get()`, `Log::info()`, `DB::table()`, `Http::get()`. These are **facades** — a static proxy to an object resolved from the container. `Cache::get('k')` is really "resolve the cache service from the container and call `get('k')` on it." They are not global static state; they are syntactic sugar over container resolution, and they are fully mockable in tests (`Cache::fake()`).

PHP

```
use Illuminate\Support\Facades\Cache;

Cache::put('user:1', $user, 60); // facade – looks static, isn't
$same = app('cache')->put('user:1', $user, 60); // equivalent, explicit resolve
$also = app(\Illuminate\Contracts\Cache\Repository::class); // via the contract
```

Contracts are the framework's interfaces (`Illuminate\Contracts\...`). Type-hinting a contract instead of a facade gives you constructor-injected, explicitly-declared dependencies — the more testable, Nest-like style many teams prefer for services.

COMMON MISTAKE

Treating facades as global singletons you can mutate freely, or scattering them deep inside domain logic. Facades are fine in controllers and glue code, but for core business services prefer constructor injection of contracts — it makes dependencies visible and tests trivial. Overusing facades produces code that is convenient to write and painful to test.

PRODUCTION TIP

`config()` values come from `config/*.php`, which read from `env()`. Call `env()` **only inside config files** — never in app code — because `php artisan config:cache` (used in production) freezes config and makes stray `env()` calls return `null`. This mirrors the Twelve-Factor rule you already follow in Node: read env once at the edge, pass config inward.

EXERCISE 5.1

Define a `PaymentGateway` contract with a `FakeGateway` and a `StripeGateway`. Bind the real one in a provider, then in a test swap it for the fake via the container. Inject it into a controller by type-hint and confirm you never wrote `new` for it. Explain why this is easier to test than the equivalent hand-wired Express setup.

Part 6 · Routing

Routing is where an Express developer feels most at home — and where Laravel's conventions save the most code. This part maps every routing feature you use in Express onto Laravel, then shows the two big multipliers Express lacks: **route model binding** and **resource routes**.

6.1 The basics, side by side

JAVASCRIPT / NODE.JS

```
// Express
const router = express.Router();
router.get("/users/:id", auth, controller.show);
router.post("/users", validate(schema), controller.store);
app.use("/api", router);
```

PHP

```
// Laravel – routes/api.php
use App\Http\Controllers\UserController;

Route::middleware('auth:sanctum')->get('/users/{id}', [UserController::class,
'show']);
Route::post('/users', [UserController::class, 'store']);
```

Routes for a JSON API live in `routes/api.php` (auto-prefixed `/api` and stateless); browser/session routes live in `routes/web.php` (with CSRF and session middleware). This split is a built-in convention — no manual `app.use('/api', ...)` mounting.

Feature	Express	Laravel
GET/POST/...	<code>router.get(path, h)</code>	<code>Route::get(\$path, \$action)</code>
Any of several	<code>router.all</code>	<code>Route::match(['get', 'post'], ...)</code>
Path param	<code>/:id</code>	<code>/{id}</code>
Optional param	<code>/:id?</code>	<code>/{id?}</code>
Handler	function ref	<code>[Controller::class, 'method']</code>
Middleware	<code>auth arg</code>	<code>->middleware('auth')</code>
Group/prefix	<code>app.use('/api', r)</code>	<code>Route::prefix('api')->group(...)</code>
Named route	<i>(manual)</i>	<code>->name('users.show')</code>

6.2 Parameters, constraints, and query strings

Path parameters arrive as method arguments (in order). Query-string parameters are read from the request object, exactly like `req.query`.

PHP

```
// GET /users/42?include=orders
Route::get('/users/{id}', function (Request $request, string $id) {
    $include = $request->query('include'); // like req.query.include
    $page = $request->integer('page', 1); // typed accessor with default
    // ...
})->whereNumber('id'); // constraint: id must be numeric
```

Constraints replace the regex you'd inline in Express (`/:id(\\d+)/`):

PHP

```
Route::get('/posts/{slug}', [PostController::class, 'show'])
    ->where('slug', '[a-z0-9-]+');
Route::get('/users/{id}', [UserController::class, 'show'])->whereNumber('id');
Route::get('/orders/{order}', ...)->whereUuid('order');
```

6.3 Route groups: shared prefix, middleware, and naming

Groups eliminate repetition. Everything Express does with nested routers and repeated middleware arguments, Laravel does with a fluent group.

PHP

```
Route::prefix('v1')
    ->middleware(['auth:sanctum', 'throttle:60,1'])
    ->name('api.v1.')
    ->group(function () {
        Route::get('/me', [ProfileController::class, 'show'])->name('me');
        Route::apiResource('projects', ProjectController::class);
        Route::apiResource('projects.tasks', TaskController::class); // nested
    });
```

JAVASCRIPT / NODE.JS

```
// Express equivalent – repeat auth+throttle on every subrouter, mount manually
const v1 = express.Router();
v1.use(auth, throttle);
v1.get("/me", profile.show);
// ...mount projects and nested tasks routers by hand...
app.use("/v1", v1);
```

Named routes (`->name('users.show')`) let you generate URLs by name rather than hardcoding paths — `route('users.show', $user)` — so changing a URL never breaks links. Express has no built-in equivalent; you hardcode strings.

6.4 Route Model Binding — the feature you'll miss most going back

This is the single biggest routing upgrade over Express. Type-hint an Eloquent model in your controller and Laravel **fetches it from the database by the route parameter automatically**, returning a 404 if it does not exist. You delete a whole category of "look up the entity, check for null, 404" boilerplate.

JAVASCRIPT / NODE.JS

```
// Express: you fetch and null-check by hand, every handler
router.get("/projects/:id", async (req, res) => {
  const project = await prisma.project.findUnique({ where: { id: +req.params.id } });
  if (!project) return res.status(404).json({ error: "Not found" });
  res.json(project);
});
```

PHP

```
// Laravel: {project} + Project type-hint -> auto-fetched or 404
Route::get('/projects/{project}', [ProjectController::class, 'show']);

public function show(Project $project): JsonResponse // already loaded from DB
{
    return response()->json($project); // 404 handled for you
}
```

By default it binds on the primary key. Bind on another column (e.g. a slug) by declaring it in the route or the model:

PHP

```
Route::get('/posts/{post:slug}', [PostController::class, 'show']); // WHERE slug
= ...
```

Nested bindings can be **scoped** so `/projects/{project}/tasks/{task}` verifies the task actually belongs to that project — automatic authorization-by-hierarchy you'd write by hand in Express.

PRODUCTION TIP

Route model binding + policies (Part 10) means a `show` method can be just `return $project;` — fetching, 404, and "can this user see it?" all handled declaratively before your method runs. Lean into it; resist the urge to re-fetch inside the controller.

6.5 Resource routes — CRUD in one line

`Route::apiResource` generates the seven (for `apiResource`, five) conventional CRUD routes mapped to conventionally-named controller methods. This replaces the five separate `router.get/post/put/delete` lines you write per resource in Express.

PHP

```
Route::apiResource('projects', ProjectController::class);
```

expands to:

Verb	URI	Controller method	Name
GET	<code>/projects</code>	<code>index</code>	<code>projects.index</code>
POST	<code>/projects</code>	<code>store</code>	<code>projects.store</code>
GET	<code>/projects/{project}</code>	<code>show</code>	<code>projects.show</code>
PUT/PATCH	<code>/projects/{project}</code>	<code>update</code>	<code>projects.update</code>
DELETE	<code>/projects/{project}</code>	<code>destroy</code>	<code>projects.destroy</code>

JAVASCRIPT / NODE.JS

```
// Express: the same thing, written out every time
router.get("/projects", ctrl.index);
router.post("/projects", ctrl.store);
router.get("/projects/:project", ctrl.show);
router.put("/projects/:project", ctrl.update);
router.delete("/projects/:project", ctrl.destroy);
```

Generate the matching controller with `php artisan make:controller ProjectController --api --model=Project` and the method stubs (with route model binding) are scaffolded for you.

WHEN NOT TO USE RESOURCE ROUTES

When your endpoints are not CRUD-shaped. A `POST /projects/{project}/archive` action or an RPC-ish `/reports/generate` is clearer as an explicit route to a single-action controller (`__invoke`) than as a contorted "resource." Convention where it fits; explicit routes where it does not.

6.6 Route caching and listing

SHELL

```
php artisan route:list           # inspect every route (like a route map)
php artisan route:list --path=projects # filter
php artisan route:cache         # compile routes for production speed
php artisan route:clear        # drop the cache after changes
```

COMMON MISTAKE

Running `route:cache` in production (good) but forgetting to re-run it after changing routes, so old routes are served. Also: `route:cache` requires all routes use controller references (not closures). Bake `route:cache` into your deploy step, never run it during local development.

EXERCISE 6.1

Build a versioned API group (`v1`) protected by `auth:sanctum` and `throttle` . Add `apiResource('projects')` and nested `apiResource('projects.tasks')` with scoped bindings. Add one custom action `POST /projects/{project}/archive` to an invocable controller. Run `route:list` and confirm the generated names and URIs match the table above.

Part 7 · Database & Eloquent (vs Prisma)

You model data in Node with Prisma: a schema file, generated types, and an explicit, functional query API (`prisma.user.findMany({ where, include })`). Eloquent is Laravel's ORM, and it takes the opposite philosophical stance: it is **Active Record** — each model *is* a database row with behavior, and queries read like sentences (`User::where('active', true)->get()`). Neither is "right," but the mental shift is real, so this chapter leads with it.

7.1 The core philosophical difference

Aspect	Prisma (Data Mapper-ish)	Eloquent (Active Record)
Schema source	<code>schema.prisma</code> (single file)	migrations + model classes
Types	generated at build time	dynamic (via <code>@property</code> docblocks)
A record is...	a plain object (POJO)	a model instance <i>with methods</i>
Save	<code>prisma.user.update(...)</code>	<code>\$user->save()</code> (row knows how)
Relations	<code>include: { posts: true }</code>	<code>\$user->posts</code> / <code>->with('posts')</code>
Query style	object/functional	fluent chain
Migrations	<code>prisma migrate</code> from schema diff	hand-written <code>up()</code> / <code>down()</code>

NODE.JS MENTAL MODEL

Prisma keeps data (rows) and behavior (your services) separate; a `User` is just data. Eloquent fuses them: a `User` model both *is* the row and *has* methods (`$user->posts()`, `$user->markVerified()`). If that feels like mixing concerns, you are not wrong — Part 12 covers how to keep business logic out of models. For now, embrace Active Record for data access.

7.2 Migrations — schema as versioned code

Prisma derives migrations by diffing `schema.prisma`. Laravel migrations are **explicit, hand-written** up/down classes — more verbose, but you control exactly what SQL runs, which matters for zero-downtime changes.

PHP

```
// database/migrations/2026_01_01_000000_create_projects_table.php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
```

```

return new class extends Migration {
    public function up(): void
    {
        Schema::create('projects', function (Blueprint $table) {
            $table->id(); // bigint PK, auto-inc
            $table->foreignId('owner_id')->constrained('users')->cascadeOnDelete();
            $table->string('name');
            $table->string('status')->default('active')->index();
            $table->timestamps(); // created_at, updated_at
            $table->softDeletes(); // deleted_at (optional)
            $table->index(['owner_id', 'status']); // composite index
        });
    }

    public function down(): void { Schema::dropIfExists('projects'); }
};

```

PRISMA

```

// Prisma equivalent
model Project {
  id      Int      @id @default(autoincrement())
  ownerId Int
  owner   User      @relation(fields: [ownerId], references: [id], onDelete: Cascade)
  name    String
  status  String    @default("active")
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  @@index([ownerId, status])
}

```

SHELL

```

php artisan make:migration create_projects_table # scaffold
php artisan migrate                             # apply pending
php artisan migrate:rollback                     # undo last batch
php artisan migrate:fresh --seed                 # drop all, re-run, seed

```

PRODUCTION TIP

Never edit an already-shipped migration — add a new one, exactly like you would never rewrite Git history that others have pulled. In production run `php artisan migrate --force`. For zero-downtime, split risky changes: add a nullable column in one deploy, backfill via a job, enforce `NOT NULL` in a later deploy (Part 20).

7.3 Models, mass assignment, casts, and enums

PHP

```

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

final class Project extends Model
{
    use SoftDeletes;

    protected $fillable = ['name', 'status'];      // mass-assignable (Part 11!)
    // or: protected $guarded = [];                // opposite: all but these

    protected function casts(): array
    {
        return [
            'status'      => ProjectStatus::class, // cast to a PHP enum
            'settings'    => 'array',              // JSON column <-> PHP array
            'launched_at' => 'immutable_datetime', // Carbon date object
            'is_public'   => 'boolean',
        ];
    }
}

```

Casts are the equivalent of Prisma's typed fields plus JSON handling: they convert raw DB values to rich PHP types (enums, `Carbon` dates, arrays, value objects) on read and back on write. Backing a `status` column with a `ProjectStatus` enum gives you the exhaustiveness and safety from Part 1 directly on your model.

7.4 CRUD with the query builder and Eloquent

JAVASCRIPT / NODE.JS

```

// Prisma
const active = await prisma.project.findMany({
  where: { status: "active", ownerId: 1 },
  orderBy: { createdAt: "desc" },
  take: 20,
});
const p = await prisma.project.create({ data: { name: "X", ownerId: 1 } });
await prisma.project.update({ where: { id: 5 }, data: { name: "Y" } });
await prisma.project.delete({ where: { id: 5 } });

```

PHP

```

// Eloquent
$active = Project::query()
    ->where('status', 'active')
    ->where('owner_id', 1)
    ->latest()                // orderBy created_at desc
    ->limit(20)

```

```

->get(); // -> Collection of Project models

$p = Project::create(['name' => 'X', 'owner_id' => 1]);
$p->update(['name' => 'Y']); // or: $p->name = 'Y'; $p->save();
$p->delete();

$one = Project::find(5); // by PK, or null
$oneOr = Project::findOrFail(5); // throws -> auto 404 in HTTP context
$first = Project::where('name','X')->first();

```

Prisma	Eloquent
<code>findUnique({where:{id}})</code>	<code>find(\$id) / findOrFail(\$id)</code>
<code>findMany({where})</code>	<code>where(...)->get()</code>
<code>findFirst</code>	<code>first()</code>
<code>create({data})</code>	<code>create([...])</code>
<code>update({where,data})</code>	<code>\$m->update([...])</code>
<code>upsert</code>	<code>updateOrCreate(\$attrs, \$values)</code>
<code>delete</code>	<code>\$m->delete()</code>
<code>count</code>	<code>count()</code>
<code>aggregate</code>	<code>sum()/avg()/max()</code>
<code>groupBy</code>	<code>groupBy()->selectRaw(...)</code>

7.5 Relationships

Relationships are declared as **methods** on the model. Accessing them as a **property** runs the query and caches the result; calling them as a **method** returns a query builder you can extend.

PHP

```

final class User extends Model {
    public function projects(): HasMany { return $this->hasMany(Project::class); }
    public function profile(): HasOne { return $this->hasOne(Profile::class); }
    public function roles(): BelongsToMany { return $this->belongsToMany(Role::class); }
}
final class Project extends Model {
    public function owner(): BelongsTo { return $this->belongsTo(User::class, 'owner_id'); }
    public function tasks(): HasMany { return $this->hasMany(Task::class); }
}

```

PHP

```

$user->projects; // Collection (property = run + cache)
$user->projects()->where('status','active')->get(); // method = extend query

```



```
$project->owner->name;           // belongsTo traversal
$user->roles()->attach($roleId);  // many-to-many pivot insert
$user->roles()->sync([1, 2, 3]);  // replace the set
```

Relationship	Eloquent	Prisma
One-to-one	<code>hasOne</code> / <code>belongsTo</code>	<code>@relation</code> 1-1
One-to-many	<code>hasMany</code> / <code>belongsTo</code>	<code>Post[]</code> / back-relation
Many-to-many	<code>belongsToMany</code> (pivot table)	implicit/explicit <code>@relation</code> join
Has-many-through	<code>hasManyThrough</code>	nested relation traversal
Polymorphic	<code>morphMany</code> / <code>morphTo</code>	(no native equivalent; manual)

Polymorphic relations (e.g. `comments` that belong to either a `Post` or a `Video`) are a genuine Eloquent superpower Prisma lacks natively — one `commentable_id` + `commentable_type` pair lets one table relate to many:

PHP

```
final class Comment extends Model {
    public function commentable(): MorphTo { return $this->morphTo(); }
}
final class Post extends Model {
    public function comments(): MorphMany { return $this->morphMany(Comment::class,
    'commentable'); }
}
```

7.6 Eager loading and the N+1 problem

This is the highest-value performance topic in the whole ORM. Accessing a relation as a property is **lazy** — it lazily fires a query *per model*, which inside a loop becomes the N+1 problem you already know from Prisma.

PHP

```
// N+1: 1 query for projects + 1 query PER project for its owner
$projects = Project::all();
foreach ($projects as $p) { echo $p->owner->name; } // BAD: 1 + N queries
```

PHP

```
// Eager load: 2 queries total, regardless of N (like Prisma `include`)
$projects = Project::with('owner')->get();
foreach ($projects as $p) { echo $p->owner->name; } // GOOD

Project::with(['owner', 'tasks.assignee'])->get(); // nested eager load
Project::withCount('tasks')->get();               // ->tasks_count without
loading
```

```
User::with(['projects' => fn ($q) => $q->where('status','active')])->get(); //
constrained
```

JAVASCRIPT / NODE.JS

```
// Prisma equivalent
const projects = await prisma.project.findMany({
  include: { owner: true, tasks: { include: { assignee: true } } },
});
```

COMMON MISTAKE

The N+1 query is the number-one performance bug in Eloquent apps, and it hides because lazy loading *just works* in development with 5 rows. Two defenses: (1) eager-load with `->with(...)` whenever you will touch a relation in a loop; (2) in `AppServiceProvider::boot()` call `Model::preventLazyLoading(! app()->isProduction())` so any accidental lazy load **throws in dev/test** and you catch it before shipping.

PRODUCTION TIP

Install Laravel Telescope or the Debugbar locally to see the query count per request. A page that issues 200 queries almost always has an N+1; add the right `->with(...)` and it drops to 3. This is the fastest performance win you will ever ship.

7.7 Scopes, accessors, and mutators

Local scopes are reusable query fragments — named `where` clauses (like a Prisma helper that returns a partial `where`):

PHP

```
public function scopeActive(Builder $q): Builder { return $q-
>where('status','active'); }
public function scopeOwnedBy(Builder $q, User $u): Builder { return $q-
>where('owner_id',$u->id); }

Project::active()->ownedBy($user)->latest()->get(); // reads like a sentence
```

Accessors/mutators compute or transform attributes (like Prisma client extensions or a getter on your service layer):

PHP

```
protected function fullName(): Attribute
{
    return Attribute::make(
        get: fn () => "{$this->first} {$this->last}", // $model->full_name
        set: fn ($value) => ['first' => explode(' ', $value)[0]],
    );
}
```

```
);
}
```

7.8 Pagination, transactions, and locking

Pagination is built in and returns a paginator with metadata + links — no manual `LIMIT/OFFSET` and count queries:

PHP

```
$page = Project::active()->paginate(20); // ?page=2 read automatically
// JSON: { data: [...], meta: { current_page, last_page, total }, links: {...} }
$cursor = Project::orderBy('id')->cursorPaginate(20); // keyset pagination (fast)
```

Transactions and **row locking** handle concurrency — the equivalent of Prisma's `$transaction` and `SELECT ... FOR UPDATE`:

PHP

```
use Illuminate\Support\Facades\DB;

DB::transaction(function () use ($order) {
    $account = Account::where('id', $order->account_id)
        ->lockForUpdate()           // pessimistic lock: SELECT ... FOR UPDATE
        ->firstOrFail();
    $account->decrement('balance', $order->total);
    $order->update(['status' => 'paid']);
});                               // commits on success, rolls back on exception
```

Concern	Eloquent	Prisma
Transaction	<code>DB::transaction(fn)</code>	<code>prisma.\$transaction([...])</code>
Pessimistic lock	<code>->lockForUpdate()</code>	<code>SELECT FOR UPDATE (raw)</code>
Shared lock	<code>->sharedLock()</code>	<code>raw</code>
Optimistic lock	version column + check	version column + check
Raw query	<code>DB::select('... ?', [\$x])</code>	<code>prisma.\$queryRaw</code>

WHEN NOT TO REACH FOR RAW SQL

Eloquent covers 95% of queries clearly. Drop to `DB::raw / selectRaw` for genuine reporting aggregates, window functions, or hot-path optimizations — not to avoid learning the builder. And **always** bind parameters (`DB::select('... where id = ?', [$id])`); never interpolate user input into SQL strings (Part 11).

EXERCISE 7.1

Model `User` `hasMany` `Project` `hasMany` `Task`, with `Task` `belongsTo` an assignee (a `User`). Write a query that loads all active projects for a user with their tasks and each task's assignee in **3 total queries** (prove it with the query log). Add an `active` scope, a `status` enum cast, and a `paginate(15)`. Then intentionally trigger an N+1 and fix it, noting the query-count before and after.

Part 8 · A Complete CRUD Feature

Parts 5–7 gave you the pieces; this part assembles them into one complete, production-shaped CRUD feature so you can see **where every piece belongs**. We build `Projects` (owned by a user) with `Tasks`. This is the skeleton you will copy for every resource you ever build.

8.1 The anatomy of a Laravel CRUD feature

DIAGRAM

<code>routes/api.php</code>	– <code>Route::apiResource('projects', ...)</code>
<code>app/Models/Project.php</code>	– table mapping, relations, casts, scopes
<code>database/migrations/...</code>	– schema
<code>database/factories/...</code>	– fake data for tests/seeds
<code>app/Http/Requests/...</code>	– validation (<code>StoreProjectRequest</code> , <code>UpdateProjectRequest</code>)
<code>app/Http/Controllers/...</code>	– thin orchestration (<code>index/store/show/update/destroy</code>)
<code>app/Http/Resources/...</code>	– response shape (<code>ProjectResource</code>)
<code>app/Policies/ProjectPolicy.php</code>	– authorization (Part 10)
<code>app/Actions/...</code> (optional)	– business logic when it grows (Part 12)
<code>tests/Feature/ProjectTest.php</code>	– feature tests (Part 18)

The guiding principle for Node developers: **controllers stay thin**. Validation lives in a Form Request, response shaping lives in a Resource, authorization lives in a Policy, and non-trivial business logic lives in an Action or service. The controller just wires them together — much like a clean Express handler that delegates to middleware and services.

8.2 Validation: Form Requests (your Zod/Joi equivalent)

A **Form Request** is a class that validates and authorizes a request *before* your controller runs. It is Laravel's answer to a Zod schema plus an auth guard, and its rules are declarative.

JAVASCRIPT / NODE.JS

```
// Express + Zod
const schema = z.object({
  name: z.string().min(2).max(120),
  status: z.enum(["active", "paused"]).default("active"),
});
router.post("/projects", (req, res) => {
  const parsed = schema.safeParse(req.body);
  if (!parsed.success) return res.status(422).json(parsed.error.format());
  // ...
});
```

PHP

```
// app/Http/Requests/StoreProjectRequest.php
final class StoreProjectRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()->can('create', Project::class); // Policy check
    }

    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'min:2', 'max:120'],
            'status' => ['sometimes', Rule::enum(ProjectStatus::class)],
        ];
    }
}
```

When validation fails, Laravel automatically returns a **422** with a structured error body — you never write that plumbing. Type-hint the Form Request in the controller and it runs before your method body.

8.3 Response shaping: API Resources (your serializer)

An **API Resource** decouples your database columns from your JSON output — the equivalent of a DTO/serializer that stops you leaking internal fields.

PHP

```
// app/Http/Resources/ProjectResource.php
final class ProjectResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'status' => $this->status->value,
            'owner' => UserResource::make($this->whenLoaded('owner')),
            'taskCount' => $this->whenCounted('tasks'),
            'createdAt' => $this->created_at->toIso8601String(),
        ];
    }
}
```

`whenLoaded` / `whenCounted` include a field **only if** the relation was eager- loaded — so your serializer never triggers a hidden N+1. This is a subtle but important safety net Prisma's manual `select` gives you differently.

8.4 The controller: thin orchestration

PHP

```

final class ProjectController extends Controller
{
    public function index(Request $request): AnonymousResourceCollection
    {
        $projects = Project::query()
            ->where('owner_id', $request->user()->id)
            ->with('owner')->withCount('tasks')
            ->latest()
            ->paginate(20);

        return ProjectResource::collection($projects);           // paginated JSON
    }

    public function store(StoreProjectRequest $request): JsonResponse
    {
        $project = $request->user()->projects()->create($request->validated());

        return ProjectResource::make($project)
            ->response()->setStatusCode(201);                     // 201 Created
    }

    public function show(Project $project): ProjectResource      // route model binding
    {
        $this->authorize('view', $project);                       // Policy (Part 10)
        return ProjectResource::make($project->load('owner')->loadCount('tasks'));
    }

    public function update(UpdateProjectRequest $request, Project $project):
ProjectResource
    {
        $project->update($request->validated());
        return ProjectResource::make($project);
    }

    public function destroy(Project $project): Response
    {
        $this->authorize('delete', $project);
        $project->delete();
        return response()->noContent();                           // 204
    }
}

```

Notice how much is *not* here: no manual validation, no manual 404, no manual fetch-by-id, no manual serialization loop. Compare with the Express version, where each handler repeats all of that.

8.5 Factories and seeders — fake data as first-class

Factories generate realistic fake models for tests and local seeding — like a typed fixture builder you'd assemble by hand in Node.

PHP

```
// database/factories/ProjectFactory.php
final class ProjectFactory extends Factory
{
    public function definition(): array
    {
        return [
            'owner_id' => User::factory(),
            'name'      => fake()->words(3, true),
            'status'    => ProjectStatus::Active,
        ];
    }
    public function paused(): static
    {
        return $this->state(fn () => ['status' => ProjectStatus::Paused]);
    }
}
```

PHP

```
Project::factory()->count(10)->create(); // 10 projects
Project::factory()->paused()->for($user, 'owner')->create();
User::factory()->has(Project::factory()->count(3))->create(); // graph of data
```

PRODUCTION TIP

Factories are the backbone of fast, isolated feature tests (Part 18): `Project::factory()->create()` gives every test a clean, valid record with zero fixtures to maintain. This is a meaningful ergonomics win over hand-rolling Prisma seed objects in Node test suites.

EXERCISE 8.1

Build the full `Tasks` resource nested under projects (`projects.tasks`): migration, model + relation, `StoreTaskRequest`, `TaskResource`, an `--api` controller with all five methods, and a factory. Ensure `index` eager-loads the assignee, returns paginated resources, and that creating a task auto-associates it with the project from the route. Confirm a missing project yields 404 and invalid input yields 422 — without writing either check yourself.

Part 9 · Production REST APIs

You already know REST from building Express APIs. This part is about the details that separate a toy API from a production one — consistent error envelopes, pagination/filtering/sorting, versioning, idempotency, and webhooks — and how Laravel gives you most of them declaratively.

9.1 Status codes and HTTP verbs — the contract

Action	Verb	Success code	Laravel helper
List	GET	200	<code>Resource::collection(...)</code>
Read one	GET	200 / 404	<code>findOrFail</code> → auto 404
Create	POST	201	<code>->response()->setStatusCode(201)</code>
Full update	PUT	200	<code>\$m->update(...)</code>
Partial update	PATCH	200	<code>\$m->update(...)</code>
Delete	DELETE	204	<code>response()->noContent()</code>
Validation error	—	422	Form Request (automatic)
Unauthenticated	—	401	<code>auth</code> middleware (automatic)
Forbidden	—	403	Policy (automatic)
Rate limited	—	429	<code>throttle</code> middleware (automatic)

The wins in the right column are the point: Laravel emits the correct status codes for the common failure modes **without you writing them**, where an Express app hand-rolls each `res.status(...)`.

9.2 Consistent error responses

An API must return errors in one predictable shape. In Express you build an error-handling middleware and route every error through it. In Laravel you shape exceptions centrally in `bootstrap/app.php`, and validation/auth/model errors already funnel through it.

PHP

```
// bootstrap/app.php
->withExceptions(function (Exceptions $exceptions) {
    $exceptions->render(function (Throwable $e, Request $request) {
        if (! $request->expectsJson()) return null; // let web pages render normally

        [$status, $code] = match (true) {
```

```

        $e instanceof ModelNotFoundException      => [404, 'not_found'],
        $e instanceof AuthenticationException     => [401, 'unauthenticated'],
        $e instanceof AuthorizationException      => [403, 'forbidden'],
        $e instanceof ValidationException         => [422, 'validation_failed'],
        $e instanceof ThrottleRequestsException  => [429, 'rate_limited'],
        default                                  => [500, 'server_error'],
    ];

    return response()->json([
        'error' => [
            'code'      => $code,
            'message'   => $e->getMessage(),
            'details'   => $e instanceof ValidationException ? $e->errors() : null,
        ],
    ], $status);
});
})

```

JAVASCRIPT / NODE.JS

```

// Express equivalent – a central error middleware you must remember to reach
app.use((err, req, res, next) => {
    const status = err.status ?? 500;
    res.status(status).json({ error: { code: err.code, message: err.message } });
});

```

PRODUCTION TIP

Define **domain exceptions** (`class PaymentDeclined extends DomainException`) and map them to status codes in one place. Your services `throw` meaningful errors; the central handler translates them to HTTP. This keeps business logic free of `response()` calls and gives clients a stable error contract.

9.3 Pagination, filtering, sorting, searching

Pagination is built in (Part 7). Filtering and sorting from query strings you build explicitly — resist letting clients pass arbitrary column names (an injection and info-leak risk). A safe, allow-listed pattern:

PHP

```

public function index(Request $request): AnonymousResourceCollection
{
    $sortable = ['created_at', 'name', 'status'];
    $sort = in_array($request->query('sort'), $sortable, true)
        ? $request->query('sort') : 'created_at';

    $projects = Project::query()
        ->when($request->filled('status'),
            fn ($q) => $q->where('status', $request->string('status')))

```

```

->when($request->filled('q'),
    fn ($q) => $q->where('name', 'like', '%'.$request->string('q').'%'))
->orderBy($sort, $request->query('dir') === 'asc' ? 'asc' : 'desc')
->paginate($request->integer('per_page', 20))
->withQueryString(); // keep filters in page links

return ProjectResource::collection($projects);
}

```

The `when()` conditionals are the Collection trick from Part 2 applied to a query builder — add a clause only if the parameter is present, no `if` ladders.

WHEN NOT TO HAND-ROLL FILTERING

For rich, standardized filtering/sorting/includes, the community package `spatie/laravel-query-builder` maps `?filter[status]=active&sort=-created_at` to safe, allow-listed query operations. Reach for it once your ad-hoc `when()` chains sprawl — but keep the allow-list discipline either way.

9.4 API versioning

Version via a route prefix and a namespace of controllers/resources. Keep v1 untouched while v2 evolves.

PHP

```

Route::prefix('v1')->group(base_path('routes/api_v1.php'));
Route::prefix('v2')->group(base_path('routes/api_v2.php'));

```

9.5 Idempotency and idempotency keys

`GET`, `PUT`, and `DELETE` are idempotent by definition; `POST` is not — a retried "create charge" can double-charge. The industry pattern (Stripe-style) is an **idempotency key**: the client sends a unique `Idempotency-Key` header, and the server stores the first response and replays it for retries.

PHP

```

final class EnsureIdempotent
{
    public function handle(Request $request, Closure $next): Response
    {
        $key = $request->header('Idempotency-Key');
        if (! $key) return $next($request);

        $cacheKey = "idem:{$request->user()->id}:{$key}";
        if ($cached = Cache::get($cacheKey)) {
            return response()->json($cached['body'], $cached['status']); // replay
        }
        $response = $next($request);
    }
}

```

```

        Cache::put($cacheKey, [
            'body' => json_decode($response->getContent(), true),
            'status' => $response->getStatusCode(),
        ], now()->addHours(24));
        return $response;
    }
}

```

NODE.JS MENTAL MODEL

This is exactly the idempotency middleware you'd write in Express with a Redis `SETNX` — same concept, same Redis, expressed as a Laravel middleware. Apply it via `>middleware('idempotent')` on your unsafe `POST` routes (payments, order creation).

9.6 Webhooks (receiving)

Receiving webhooks safely has three rules: **verify the signature**, **respond fast (2xx)** then **process async**, and **treat delivery as at-least-once** (dedupe). Signature verification is covered in Part 11; the async pattern uses a queued job (Part 13).

PHP

```

public function handle(Request $request): Response
{
    // 1. verify signature (Part 11) – reject 400 if invalid
    // 2. dedupe by event id (webhook may be delivered more than once)
    if (WebhookEvent::where('provider_id', $request->input('id'))->exists()) {
        return response()->noContent(); // already handled, ack anyway
    }
    // 3. persist + queue, return fast
    $event = WebhookEvent::create([...]);
    ProcessWebhook::dispatch($event); // do the slow work off-request
    return response()->noContent(); // 204 immediately
}

```

COMMON MISTAKE

Doing the webhook's real work synchronously inside the request. Providers retry aggressively on slow/failed responses, causing duplicate processing and timeouts. Always: verify → dedupe → enqueue → `204`. The heavy lifting happens on a worker.

EXERCISE 9.1

Add to your Projects API: allow-listed sorting and `status / q` filtering with `withQueryString()` pagination; a central JSON error handler mapping a custom `ProjectLocked` domain exception to `409 Conflict`; and an idempotency middleware on `POST /projects`. Write one test that retries the same `POST` with the same key and asserts a single project was created.

Part 10 · Authentication & Authorization

In Node you assemble auth from parts: `bcrypt/argon2` for hashing, `jsonwebtoken` or `express-session`, Passport strategies, and hand-written middleware/guards for permissions. Laravel ships all of this as first-class, integrated subsystems. The distinction to hold throughout: **authentication** is "who are you" (identity); **authorization** is "what may you do" (permissions). Laravel keeps them cleanly separate.

10.1 Password hashing

JAVASCRIPT / NODE.JS

```
import bcrypt from "bcrypt";
const hash = await bcrypt.hash(password, 12);
const ok = await bcrypt.compare(password, hash);
```

PHP

```
use Illuminate\Support\Facades\Hash;
$hash = Hash::make($password);           // bcrypt/argon2, cost from config
$ok    = Hash::check($password, $hash);
if (Hash::needsRehash($hash)) { /* upgrade cost transparently */ }
```

The `password` cast on the `User` model even hashes automatically on assignment. Never store or log plaintext; never roll your own hashing.

10.2 Two auth worlds: sessions vs API tokens

Laravel has two authentication surfaces, and choosing correctly is the main decision:

Use case	Node analog	Laravel tool
Server-rendered / same-site SPA	<code>express-session</code> + cookie	Session guard (<code>web</code>)
First-party SPA/mobile (cookie)	session cookie	Sanctum (SPA mode)
API tokens (PATs, mobile, CLI)	JWT / opaque token	Sanctum (token mode)
OAuth2 server (issue tokens to 3rd parties)	own OAuth server	Passport

For 90% of APIs, the answer is **Sanctum**. It issues simple, revocable, database-backed API tokens (opaque, not JWT) and also supports cookie-based SPA auth. Passport is the heavier OAuth2 server you only need when *other* companies get tokens to your API.

10.3 Sanctum API tokens in practice

PHP

```
// Issue a token (e.g. in a login controller)
public function login(LoginRequest $request): JsonResponse
{
    $user = User::where('email', $request->string('email'))->first();
    if (! $user || ! Hash::check($request->string('password'), $user->password)) {
        throw ValidationException::withMessages(['email' => 'Invalid credentials']);
    }
    $token = $user->createToken('mobile', ['projects:read', 'projects:write']);
    return response()->json(['token' => $token->plainTextToken]);
}
```

PHP

```
// Protect routes – the client sends: Authorization: Bearer <token>
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/me', fn (Request $r) => $r->user());
    Route::apiResource('projects', ProjectController::class);
});
```

Tokens are stored hashed in `personal_access_tokens`, can carry **abilities** (scopes), and are revocable (`$user->tokens()->delete()` = logout everywhere). The abilities map onto Sanctum's `->tokenCan('projects:write')` checks.

JAVASCRIPT / NODE.JS

```
// The Node equivalent you'd assemble by hand
const token = jwt.sign({ sub: user.id, scopes }, SECRET, { expiresIn: "30d" });
app.use("/api", (req, res, next) => { /* verify bearer, load user */ });
```

NODE.JS MENTAL MODEL

Sanctum's default tokens are **opaque + database-backed**, not self-contained JWTs. The tradeoff is the classic one you know: JWTs are stateless and can't be revoked before expiry; DB tokens require a lookup but are instantly revocable. Sanctum picks revocability by default, which is usually the right call for first-party apps.

10.4 Authorization: Gates and Policies

Once you know *who* the user is, **Gates** and **Policies** decide *what* they can do — Laravel's structured replacement for scattered `if (user.role === ...)` checks.

- A **Gate** is a standalone closure for a simple ability (`viewDashboard`).
- A **Policy** is a class of methods for a model (`view`, `update`, `delete` on `Project`) — the idiomatic choice for resource permissions.

PHP

```
final class ProjectPolicy
{
    public function view(User $user, Project $project): bool
    {
        return $project->owner_id === $user->id || $user->isAdmin();
    }
    public function update(User $user, Project $project): bool
    {
        return $project->owner_id === $user->id;
    }
    public function delete(User $user, Project $project): bool
    {
        return $this->update($user, $project) && ! $project->isLocked();
    }
}
```

You then enforce policies from three places, all reading the same rules:

PHP

```
$this->authorize('update', $project);           // in a controller (throws 403)
$user->can('delete', $project);                 // boolean check anywhere
Gate::authorize('view', $project);             // via the Gate facade
// or declaratively in a route:
Route::put('/projects/{project}', ...) -> middleware('can:update,project');
```

Because a Form Request's `authorize()` (Part 8) can call `can()`, authorization runs **before** your controller body — clean and centralized.

10.5 Roles and permissions at scale

For simple apps, boolean/enum roles on the user plus policies suffice. For role-and-permission matrices (admin/editor/viewer with granular permissions), the near-universal package is `spatie/laravel-permission`, which adds `$user->assignRole('editor')`, `$user->givePermissionTo('projects.delete')`, and `$user->can('projects.delete')` backed by tables — the equivalent of building an RBAC layer in Node, but maintained and battle-tested.

10.6 Email verification & password reset

Both are built in. `MustVerifyEmail` on the `User` model plus the `verified` middleware gates routes behind a verified email; the password-reset flow (token table, throttled emails, reset endpoint) ships as scaffolding. In Node you'd wire nodemailer, a tokens table, and expiry logic by hand; here it is configuration plus a notification.

COMMON MISTAKE

Confusing authentication with authorization and stuffing permission checks into middleware only. Middleware is great for "must be logged in" (`auth:sanctum`) and coarse gates, but per-record rules ("can THIS user edit THIS project") belong in a **Policy**, where they are testable and reused by controllers, Form Requests, and even Blade/JSON conditionals.

PRODUCTION TIP

Give tokens the **minimum abilities** they need and check them (`$request->user()->tokenCan('projects:write')`). Set token expirations for mobile/CLI use, provide a "revoke all sessions" action (`$user->tokens()->delete()`), and rate-limit the login route hard (Part 11) to blunt credential stuffing.

EXERCISE 10.1

Implement Sanctum login issuing a scoped token; protect the Projects API with `auth:sanctum`; write a `ProjectPolicy` enforcing owner-only update/delete (admins exempt) and call it from both the controller and the `UpdateProjectRequest`. Add a test proving user B gets **403** editing user A's project and **404** for a nonexistent one.

Part 11 · Security

Security is where framework conventions earn their keep: Laravel closes many holes by default that a hand-assembled Express app leaves open. This part is a practical checklist — each item names the threat, your Node habit, and the Laravel-specific control.

11.1 SQL injection

Eloquent and the query builder **parameterize everything** by default — you get prepared statements for free. The only way to introduce SQL injection is to hand-concatenate user input into a raw expression.

PHP

```
User::where('email', $request->input('email'))->first(); // SAFE: bound param
DB::select('select * from users where email = ?', [$email]); // SAFE: bound
DB::select("select * from users where email = '$email'"); // DANGER: injection
$q->orderBy($request->input('sort')); // DANGER: column not bound
```

COMMON MISTAKE

Column names and `orderBy` directions are **not** bindable parameters, so passing user input straight into `orderBy()` / `selectRaw()` is injectable even though values are safe. Always allow-list column names (Part 9) and bind values with `?` placeholders in any raw SQL.

11.2 Mass assignment

This is a PHP-specific footgun with no direct Express analog. `Model::create($request->all())` will set **any** column present in the payload — including `is_admin` or `owner_id` if an attacker adds them. The `$fillable` / `$guarded` allow-list on the model is your guard, and you should pair it with validated input, not raw request data.

PHP

```
protected $fillable = ['name', 'status']; // ONLY these are mass-assignable
Project::create($request->validated()); // validated() ⊆ known fields
Project::create($request->all()); // risky: passes unknown keys through
```

PRODUCTION TIP

Two-layer defense: (1) `$fillable` on every model; (2) always create/update from `$request->validated()` (Form Request output), never `$request->all()`. The Form Request defines the exact accepted fields; mass-assignment protection is the backstop if one slips through.

11.3 CSRF and CORS

- **CSRF:** Session-based `web` routes are CSRF-protected automatically (token in forms/cookies). **Stateless API routes using bearer tokens do not need CSRF** protection — a token in an `Authorization` header can't be sent by a browser cross-site the way a cookie can. If you use Sanctum's *cookie* SPA mode, CSRF applies and Sanctum handles it.
- **CORS:** Configured in `config/cors.php` (allowed origins/methods/headers). Set explicit origins in production — never reflect `*` with credentials.

PHP

```
// config/cors.php
'allowed_origins' => ['https://app.example.com'], // explicit, not '*'
'supports_credentials' => true,                // only if using cookie auth
```

11.4 XSS and output encoding

For JSON APIs, XSS is largely the client's concern, but if you render Blade templates, `{{ $value }}` **auto-escapes** HTML (safe by default); `{!! $raw !!}` does **not** (dangerous). Never pass user content through `{!! !!}` without sanitizing.

11.5 Rate limiting

Built-in, Redis-backed, and declarative — the equivalent of `express-rate-limit` but integrated with the auth and response system (auto 429 + `Retry-After`).

PHP

```
// bootstrap/app.php or a service provider
RateLimiter::for('api', fn (Request $r) =>
    Limit::perMinute(60)->by($r->user()?->id ?: $r->ip()));

RateLimiter::for('login', fn (Request $r) =>
    Limit::perMinute(5)->by($r->ip())); // blunt credential stuffing

Route::middleware('throttle:api')->group(fn () => /* ... */);
Route::post('/login', ...)->middleware('throttle:login');
```

11.6 Encryption, secrets, and signed URLs

- **Secrets** live in `.env` (gitignored) and are read through `config()`. The `APP_KEY` powers encryption and must be stable across servers.
- **Encryption at rest** for sensitive columns: the `encrypted` cast transparently encrypts/decrypts (`'ssn' => 'encrypted'`), or use `Crypt::encryptString()`.
- **Signed URLs** produce tamper-proof, optionally-expiring links without a DB record — perfect for email confirmations and temporary access:

PHP

```
URL::temporarySignedRoute('unsubscribe', now()->addDays(3), ['user' => $user->id]);
// the route: ->middleware('signed') rejects tampered/expired links automatically
```

11.7 File upload security

PHP

```
$request->validate([
    'avatar' => ['required', 'file', 'mimes:jpg,png,webp', 'max:2048'], // 2 MB
]);
$path = $request->file('avatar')->store('avatars', 's3'); // never trust client
name
```

Validate MIME type and size; store under a generated name (Laravel does this); never execute or serve uploads from a path where PHP could run them; put private files on a private disk and hand out **temporary signed URLs** (Part 17).

11.8 Webhook signature verification

Inbound webhooks must be authenticated by verifying the provider's HMAC signature against the **raw** body, using a **timing-safe** comparison.

PHP

```
$payload    = $request->getContent(); // RAW body, not parsed
$expected   = hash_hmac('sha256', $payload, config('services.stripe.webhook_secret'));
$provided   = $request->header('Stripe-Signature');
if (! hash_equals($expected, $provided)) { // timing-safe compare
    abort(400, 'Invalid signature');
}
```

COMMON MISTAKE

Verifying against the *parsed/re-encoded* body instead of the raw bytes (the signature won't match), or using `== / ===` instead of `hash_equals` (leaks timing). Both are classic webhook bugs. Capture the raw body before any middleware reformats it.

11.9 The pre-flight security checklist

Control	Default?	Your job
SQL injection	safe by default	never concatenate raw SQL; allow-list columns
Mass assignment	opt-in guard	set <code>\$fillable</code> ; use <code>validated()</code>
Password hashing	bcrypt/argon2	use <code>Hash</code> ; never log plaintext
CSRF (web)	on	leave on; APIs use bearer tokens

Control	Default?	Your job
CORS	configurable	explicit origins in prod
Rate limiting	available	throttle API + login routes
Secrets	<code>.env</code>	gitignore; rotate; <code>config:cache</code> in prod
HTTPS	infra	force via <code>URL::forceScheme('https')</code> / proxy
Auth-z per record	Policies	enforce in Form Request/controller
Dependency CVEs	—	<code>composer audit</code> in CI

WHEN NOT TO TRUST DEFAULTS BLINDLY

Defaults are strong but not magic. `$guarded = []` (allow all) plus `$request->all()` re-opens mass assignment; `'allowed_origins' => ['*']` with credentials defeats CORS; a raw `DB::statement` with interpolation reintroduces injection. Audit the places where you *left the framework's path* — that is where the holes are.

EXERCISE 11.1

Harden your Projects API: add `throttle:api` globally and `throttle:login` on auth; confirm creating a project ignores an injected `owner_id` / `is_admin` in the payload; add a signed unsubscribe route; and implement HMAC webhook verification with `hash_equals`. Run `composer audit` and resolve any advisories.

Part 12 · Application Architecture

Laravel gives you many places to put logic: controllers, models, services, actions, DTOs, events, listeners, jobs, policies, value objects. The danger for an experienced Node developer is *over-architecting* — importing DDD ceremony into an app that doesn't need it. This part is a decision guide: what each pattern is for, and, just as important, **when not to use it**.

12.1 The default: fat where it belongs, thin everywhere else

Start simple. A Laravel app is healthy when: **validation** is in Form Requests, **authorization** is in Policies, **response shape** is in Resources, **data access** is on models, and **controllers are thin orchestrators**. For simple CRUD, that is the *entire* architecture — you do not need a service layer.

PHP

```
public function store(StoreProjectRequest $request): JsonResponse
{
    $project = $request->user()->projects()->create($request->validated());
    return ProjectResource::make($project)->response()->setStatusCode(201);
}
```

If that reads clearly, **do not add a `ProjectService`**. A service class that only forwards to `Model::create()` is pure ceremony.

WHEN NOT TO CREATE A SERVICE CLASS

Do not create a service class just because you can. If a controller method is a few lines of orchestration with no branching business logic, a service adds a file, an injection, and indirection for zero benefit. Extract only when logic is (a) non-trivial, (b) reused across entry points, or (c) worth isolating for testing. Premature service layers are the most common form of Laravel over-engineering.

12.2 Services and Actions — when logic outgrows the controller

When a use case involves multiple steps (charge, create records, dispatch jobs, send notifications), extract it. Two popular shapes:

- **Service:** a class grouping related operations (`BillingService` with `charge()`, `refund()`).

- **Action:** a single-purpose invokable class for one use case (`CreateProject` , `PublishPost`) — one class, one public method. Actions keep each use case small and testable; many teams prefer them over broad services.

PHP

```
final class CreateProject
{
    public function __construct(private readonly Dispatcher $events) {}

    public function handle(User $owner, array $data): Project
    {
        return DB::transaction(function () use ($owner, $data) {
            $project = $owner->projects()->create($data);
            $project->members()->attach($owner->id, ['role' => 'owner']);
            $this->events->dispatch(new ProjectCreated($project));
            return $project;
        });
    }
}

// controller stays thin:
public function store(StoreProjectRequest $r, CreateProject $action): JsonResponse
{
    $project = $action->handle($r->user(), $r->validated());
    return ProjectResource::make($project)->response()->setStatusCode(201);
}
```

NODE.JS MENTAL MODEL

This is exactly your Node instinct: keep route handlers thin, push logic into a `services/` or `use-cases/` layer. An Action is a use-case function boxed into a class so the container can inject its dependencies. The difference from Express is only mechanical (DI via type-hints); the layering judgment is identical.

12.3 DTOs and Value Objects

A **DTO** carries structured, typed data between layers (a validated payload as an object rather than a loose array). A **Value Object** models a domain concept with invariants (`Money` , `EmailAddress`) — immutable, compared by value. `readonly` classes (Part 1) make both ergonomic.

PHP

```
final readonly class Money
{
    public function __construct(public int $cents, public string $currency = 'USD')
    {}

    public function add(Money $o): self
    {
        if ($o->currency !== $this->currency) throw new DomainException('currency
```

```

mismatch');
    return new self($this->cents + $o->cents, $this->currency);
}
}

```

The `spatie/laravel-data` package turns validated requests into typed DTOs that double as Resources — handy once loose arrays start hurting.

12.4 Events and Listeners — decoupling side effects

Events let one action announce "something happened" and **listeners** react, without the action knowing about them. This is the Observer pattern and the direct analog of Node's `EventEmitter` — but Laravel listeners can be **queued** (run on a worker) with one interface.

PHP

```

event(new ProjectCreated($project));          // fire

final class SendProjectWelcome implements ShouldQueue // handled off-request
{
    public function handle(ProjectCreated $event): void { /* send email */ }
}

```

Node	Laravel
<code>emitter.emit('created', p)</code>	<code>event(new ProjectCreated(\$p))</code>
<code>emitter.on('created', fn)</code>	a Listener class subscribed to the event
in-process, synchronous	sync or queued via <code>ShouldQueue</code>
manual wiring	auto-discovered listeners

WHEN NOT TO USE EVENTS

Events add indirection: reading the code, you can't see what happens next without hunting for listeners. Use them for genuinely decoupled, cross-cutting side effects (notifications, analytics, cache busting) — **not** for the core steps of a use case, which are clearer written inline in an Action. Over-eventing turns a linear flow into a scavenger hunt.

12.5 Repositories — usually unnecessary here

In many Node/TypeScript codebases you wrap the ORM in repository interfaces for testability and swappability. In Laravel this is **usually redundant**: Eloquent models already are the data-access layer, and the database is faked in tests with an in-memory SQLite or a transactional test DB (Part 18). A repository over Eloquent typically adds a layer that hides Eloquent's best features (scopes, eager loading) behind a thinner interface.

WHEN NOT TO ADD REPOSITORIES

Skip repositories unless you have a concrete need: multiple interchangeable data sources, a domain model deliberately isolated from persistence (heavy DDD), or a mandate to keep framework types out of the domain. For a normal Laravel API, querying Eloquent directly in Actions/controllers is idiomatic and more capable. Do not port the Node repository habit reflexively.

12.6 A pragmatic layering rule of thumb

Situation	Reach for
Simple CRUD	Controller + Form Request + Resource + Model
Multi-step use case	+ an Action (or Service)
Reused domain value with rules	+ a Value Object / DTO
Decoupled side effects	+ Events/Listeners (queued)
Slow work	+ a Job (Part 13)
Swappable integration (payments, SMS)	+ a Contract + binding (Part 5)
Multiple data sources / strict DDD	+ Repositories (rarely)

PRODUCTION TIP

Let architecture *emerge*. Ship the thin version, and extract an Action the first time a controller method grows a second responsibility or gets copied. Code that is easy to change beats code that is "correctly layered" up front. Every abstraction you add is a tax the whole team pays on every read.

EXERCISE 12.1

Take a controller method that (1) creates a record, (2) attaches a pivot, (3) sends a notification, and (4) fires analytics. Refactor it into a `CreateProject` action wrapped in a transaction, move the notification and analytics into **queued listeners** on a `ProjectCreated` event, and leave the controller at three lines. Then write one sentence justifying why the notification is an event but the pivot attach is not.

Part 13 · Queues & Background Jobs

You know this world from **BullMQ**: producers push jobs to Redis, worker processes consume them, with retries, backoff, delays, and failure handling. Laravel Queues are the same architecture with a friendlier API and deep framework integration. Because classic PHP requests cannot do background work (Part 3), queues are not optional polish here — they are *the* mechanism for anything slow: email, external API calls, PDF generation, image processing, webhooks.

13.1 The mental map

BullMQ / Node	Laravel Queue
<code>new Queue('emails')</code>	queue connection + named queue
<code>queue.add('job', data)</code>	<code>SomeJob::dispatch(\$data)</code>
<code>new Worker('emails', fn)</code>	<code>php artisan queue:work</code>
job processor function	a job class's <code>handle()</code>
<code>attempts</code> , <code>backoff</code>	<code>\$tries</code> , <code>backoff()</code>
<code>removeOnComplete</code>	default; failures → <code>failed_jobs</code> table
delayed jobs	<code>->delay(now())->addMinutes(5)</code>
<code>Queue.add</code> bulk / flows	job batches and chains
Redis (default)	Redis / SQS / database / Beanstalkd
Bull Board	Horizon dashboard

13.2 A job, dispatched and handled

JAVASCRIPT / NODE.JS

```
// BullMQ
await emailQueue.add("welcome", { userId: user.id }, { attempts: 5, backoff: 2000 });
new Worker("email", async (job) => { await sendWelcome(job.data.userId); });
```

PHP

```
// Laravel: the job class IS the processor + the payload
final class SendWelcomeEmail implements ShouldQueue
{
    use Queueable;

    public int $tries = 5;
    public int $timeout = 30;
```

```

    public function __construct(public readonly User $user) {}    // serialized
    payload

    public function handle(Mailer $mailer): void                // deps injected
    {
        $mailer->to($this->user)->send(new WelcomeMail($this->user));
    }

    public function backoff(): array { return [10, 30, 60]; }    // seconds between
    retries
}

SendWelcomeEmail::dispatch($user);                            // enqueue
SendWelcomeEmail::dispatch($user)->onQueue('emails')->delay(now()->addMinutes(5));

```

Because the job's constructor takes a `User` model, Laravel serializes only its ID and **re-fetches a fresh model** when the worker runs it (`SerializesModels`). That avoids stale data — a subtle correctness feature over passing a plain object through Redis.

13.3 Running workers (and why they are different from FPM)

SHELL

```

php artisan queue:work redis --queue=high,default --tries=3 --max-time=3600
php artisan queue:listen      # dev: reloads code each job (slower)
php artisan queue:work        # prod: long-lived, must be restarted on deploy

```

A `queue:work` process is **long-lived** — the one place classic PHP keeps memory warm across many jobs. That has a consequence Node developers must internalize in reverse: because the worker holds your booted app in memory, it does **not** see new code until restarted.

COMMON MISTAKE

Deploying new code and wondering why workers still run the old logic. You must `php artisan queue:restart` (signals workers to gracefully die after the current job; Supervisor respawns them with fresh code) on **every** deploy. This is the exact inverse of the FPM rule from Part 3 — FPM sees new code instantly, workers do not.

13.4 Retries, backoff, failures, idempotency

Failed jobs (after exhausting `$tries`) land in the `failed_jobs` table for inspection and replay — the equivalent of BullMQ's failed set.

SHELL

```
php artisan queue:failed      # list failures
php artisan queue:retry all   # requeue them
```

Because a job may run **more than once** (retry after a partial success, at-least-once delivery), jobs must be **idempotent** — the single most important discipline in any queue system, and identical to what you practice with BullMQ.

PHP

```
public function handle(): void
{
    // idempotency guard: skip if already processed
    if ($this->order->fresh()->status === 'charged') return;
    $chargeId = $this->gateway->charge($this->order->total_cents); // external
    effect
    $this->order->update(['status' => 'charged', 'charge_id' => $chargeId]);
}
```

Laravel also offers `ShouldBeUnique` (prevent duplicate concurrent jobs) and `WithoutOverlapping / RateLimited` middleware for jobs.

13.5 Chains, batches, and delays

PHP

```
// Chain: run in sequence, stop if one fails (like BullMQ flows)
Bus::chain([
    new ChargeOrder($order),
    new SendReceipt($order),
    new NotifyWarehouse($order),
])->dispatch();

// Batch: run many in parallel, with an aggregate completion callback
Bus::batch($users->map(fn ($u) => new RecomputeStats($u)))
    ->then(fn (Batch $b) => Log::info('all done'))
    ->catch(fn (Batch $b, Throwable $e) => Log::error('batch failed'))
    ->allowFailures()
    ->dispatch();
```

13.6 Monitoring with Horizon

Horizon is the Redis-queue dashboard and supervisor — throughput, wait times, failed jobs, and auto-balancing of worker processes across queues. It is the Laravel counterpart to Bull Board plus a process manager, and it is the standard way to run and observe queues in production (Part 20).

Priority pattern	How
Priority queues	<code>--queue=high,default,low</code> (drains high first)
Dedicated workers per queue	separate Supervisor programs / Horizon supervisors
Rate-limited third-party calls	<code>RateLimited</code> job middleware + Redis limiter
No overlap for a resource	<code>WithoutOverlapping(\$lockKey)</code> middleware

PRODUCTION TIP

Route jobs to named queues by urgency (`emails` , `webhooks` , `default` , `reports`) and give latency-sensitive queues more workers via Horizon. Keep a slow `reports` job off the same workers as user-facing `emails` so a batch export never delays a password-reset mail. This is the same queue-topology thinking you'd apply with BullMQ.

EXERCISE 13.1

Move welcome-email sending and an external CRM sync off the registration request into two queued jobs. Make the CRM sync idempotent (guard on an external id), give it `$tries = 5` with exponential `backoff()` , and route it to a `webhooks` queue. Force a failure, inspect `queue:failed` , fix, and `queue:retry` . Confirm that redeploying requires `queue:restart` to pick up a change to the job.

Part 14 · Task Scheduling

In Node you reach for `node-cron` inside your process, or system cron, or a hosted scheduler. The trouble with an in-process scheduler in classic PHP is Part 3 again: there is no long-lived process to hold a timer. Laravel solves this elegantly — you define the schedule **in code**, and a **single** system cron entry drives the whole thing.

14.1 One cron entry to rule them all

CRON

```
# The ONLY crontab line you add – runs the scheduler every minute
* * * * * cd /var/www/app && php artisan schedule:run >> /dev/null 2>&1
```

That single entry wakes Laravel each minute; Laravel then decides which of your scheduled tasks are due. You never add more crontab lines — new schedules are just code.

PHP

```
// routes/console.php (Laravel 11+) – define the schedule in code
use Illuminate\Support\Facades\Schedule;

Schedule::command('reports:daily')->dailyAt('02:00');
Schedule::job(new PruneOldExports)->weekly();
Schedule::call(fn () => Cache::forget('stats'))->hourly();

Schedule::command('sync:crm')
    ->everyFiveMinutes()
    ->withoutOverlapping()           // don't start if the last run is still going
    ->onOneServer()                 // in a multi-server cluster, run on just one
    ->runInBackground();
```

JAVASCRIPT / NODE.JS

```
// node-cron equivalent – but this lives INSIDE a process that must stay up
cron.schedule("0 2 * * *", () => runDailyReports());
```

14.2 The frequency vocabulary

Frequency	Method
Every minute	<code>->everyMinute()</code>
Every N minutes	<code>->everyFiveMinutes()</code> , <code>->everyThirtyMinutes()</code>
Hourly / at minute	<code>->hourly()</code> , <code>->hourlyAt(15)</code>

Frequency	Method
Daily / at time	<code>->daily()</code> , <code>->dailyAt('02:00')</code>
Weekly / monthly	<code>->weekly()</code> , <code>->monthly()</code>
Weekdays / specific day	<code>->weekdays()</code> , <code>->sundays()</code>
Raw cron	<code>->cron('* / 10 * * * *')</code>
Timezone	<code>->timezone('Africa/Cairo')</code>

14.3 Production-critical guards

Two flags prevent the classic scheduling disasters:

- `withoutOverlapping()` — if a daily report still runs when the next tick fires, don't launch a second copy (prevents pile-ups and double work).
- `onOneServer()` — in a horizontally-scaled deployment, every server runs the same cron line; this ensures a given task fires on **exactly one** of them (requires a shared cache lock, e.g. Redis).

PHP

```
Schedule::command('billing:charge-due')
    ->dailyAt('03:00')
    ->withoutOverlapping()
    ->onOneServer()
    ->emailOutputOnFailure('ops@example.com'); // alert if it errors
```

COMMON MISTAKE

Deploying to three app servers, adding the `schedule:run` cron on each, and then charging every customer **three times** because the task ran on all three. `onOneServer()` (backed by a shared Redis lock) is mandatory the moment you have more than one app server. This bug is invisible on a single box and catastrophic on a cluster.

PRODUCTION TIP

Prefer `Schedule::command(...)` (an Artisan command, Part 19) over `Schedule::call(closure)` for anything non-trivial: commands are testable in isolation, runnable on demand, and their output is loggable. Combine with `->onOneServer()` - `>withoutOverlapping()` and route heavy work into a queued job rather than doing it inline in the scheduled tick.

WHEN NOT TO SCHEDULE INLINE WORK

A scheduled task should *dispatch* work, not *do* heavy work synchronously. If `reports:daily` takes 20 minutes, don't block the scheduler tick — have the command dispatch a queued job (Part 13) so the scheduler stays snappy and the heavy lifting runs on a worker with proper retries and monitoring.

EXERCISE 14.1

Create an Artisan command `exports:prune` that deletes export files older than 30 days, schedule it daily at 02:30 with `withoutOverlapping()` and `onOneServer()`, and have it dispatch a queued job for the actual deletion. Verify locally by running `php artisan schedule:run` and `php artisan schedule:list`.

Part 15 · Caching & Redis

Recall Part 3's core lesson: classic PHP shares nothing between requests, so you **cannot** cache in a module-level variable the way you do in a long-lived Node process. Every cache in PHP is *external* — Redis, Memcached, APCu, or the database — shared across all workers. This is actually cleaner: your cache is already distributed and survives deploys, where a Node in-process `Map` cache does not.

15.1 The cache API

JAVASCRIPT / NODE.JS

```
// Node with ioredis, hand-rolled
let user = JSON.parse(await redis.get(`user:${id}`));
if (!user) { user = await db.user.find(id); await redis.setex(`user:${id}`, 3600,
JSON.stringify(user)); }
```

PHP

```
use Illuminate\Support\Facades\Cache;

Cache::put('user:1', $user, now()->addHour()); // set with TTL
$user = Cache::get('user:1', $default);        // get with fallback
Cache::forget('user:1');                       // delete
Cache::has('user:1');                          // exists?
Cache::increment('views:1');                   // atomic counter
```

15.2 Cache-aside with `remember` — the pattern you'll use most

`Cache::remember` is the cache-aside pattern in one call: return the cached value, or compute it, store it with a TTL, and return it. This replaces the get/check/set dance above.

PHP

```
$stats = Cache::remember('dashboard:stats', now()->addMinutes(10), function () {
    return [
        'users' => User::count(),
        'revenue' => Order::sum('total'),
    ]; // only runs on a cache miss
});

Cache::rememberForever('config:features', fn () => Feature::all());
```


NODE.JS MENTAL MODEL

`remember(key, ttl, fn)` is the memoize-with-TTL helper you always end up writing in Node, standardized and backed by Redis instead of a process Map. Because it's external, all your workers and servers share one cache — no per-instance cache inconsistency to reason about.

15.3 Invalidation strategies

The two hard problems remain the same as in Node: naming keys and knowing when to invalidate.

Strategy	How	When
TTL expiry	<code>remember(\$k, \$ttl, \$fn)</code>	data can be slightly stale
Explicit forget	<code>Cache::forget(\$k)</code> on write	precise invalidation
Model events	forget in a model <code>saved</code> / <code>deleted</code> observer	keep cache tied to data
Cache tags	<code>Cache::tags(['users'])->flush()</code>	group invalidation (Redis/Memcached)
Versioned keys	include a version in the key	invalidate by bumping version

PHP

```
// Invalidate on write via a model observer
protected static function booted(): void
{
    static::saved(fn (Project $p) => Cache::forget("project:{$p->id}"));
    static::deleted(fn (Project $p) => Cache::forget("project:{$p->id}"));
}
```

15.4 Atomic locks and distributed locks

Because workers and servers run concurrently, you need **distributed locks** for "only one process may do X at a time" (e.g. generating an expensive report, processing a payment). Redis-backed atomic locks provide this — the equivalent of a Redlock in Node.

PHP

```
$lock = Cache::lock('report:generate', 120); // 120s TTL safety
if ($lock->get()) {
    try { /* do the exclusive work */ }
    finally { $lock->release(); }
}

// or block up to 10s waiting to acquire, auto-release after the callback:
Cache::lock('order:'.$order->id)->block(10, function () use ($order) {
    // guaranteed exclusive section
});
```

15.5 Cache stampede (the thundering herd)

When a popular key expires, many concurrent requests miss simultaneously and all recompute the expensive value at once — a **stampede** that can crush your database. This is the same failure you guard against in Node. Defenses:

- **Locks around recompute:** `Cache::lock` so only one worker recomputes while others wait or serve stale.
- **Staggered / probabilistic TTLs:** add jitter so keys don't all expire at the same second.
- `Cache::flexible()` (Laravel 11+): serve slightly-stale data while a single request refreshes it in the background (stale-while-revalidate).

PHP

```
// stale-while-revalidate: fresh for 5 min, served-stale-and-refreshed up to 10
$value = Cache::flexible('stats', [300, 600], fn () => expensiveStats());
```

COMMON MISTAKE

Caching with a short TTL and no stampede protection on a hot endpoint, then watching the database spike every time the key expires. On high-traffic keys always pair the cache with a lock or `flexible()`. Also: choosing the `array` cache driver (per-request, wiped immediately — useless across requests) in production instead of `redis`.

WHEN NOT TO CACHE

Do not cache to paper over slow queries you could fix with an index or an eager load (Part 7). Caching adds an invalidation problem; an index does not. Cache genuinely expensive, read-heavy, tolerably-stale data — not the consequences of an N+1 you haven't fixed yet.

EXERCISE 15.1

Cache a dashboard-stats endpoint with `remember` (10-min TTL) and invalidate it via a model observer when an `Order` is created. Then simulate a stampede (many concurrent requests at expiry) and mitigate it with `Cache::flexible()` or a lock. Add a `Cache::lock` around a "generate monthly report" action so two requests can't generate it at once.

Part 16 · HTTP Clients

Calling other services is bread-and-butter backend work. In Node you reach for **Axios** (or `fetch`). Laravel wraps PHP's cURL in a fluent **HTTP client** (`Illuminate\Support\Facades\Http`) that mirrors Axios feature-for-feature, adds first-class retries, and — crucially — is trivially **fakeable in tests**.

16.1 The basics, side by side

JAVASCRIPT / NODE.JS

```
// Axios
const { data } = await axios.get("https://api.x.dev/users", {
  params: { page: 2 },
  headers: { Authorization: `Bearer ${token}` },
  timeout: 5000,
});
await axios.post("https://api.x.dev/users", { name: "Ada" });
```

PHP

```
use Illuminate\Support\Facades\Http;

$users = Http::withToken($token)
    ->timeout(5)
    ->get('https://api.x.dev/users', ['page' => 2]) // query params
    ->throw() // throw on 4xx/5xx
    ->json(); // decoded array

Http::asJson()->post('https://api.x.dev/users', ['name' => 'Ada']);
```

Axios	Laravel HTTP
<code>axios.get(url, {params})</code>	<code>Http::get(\$url, \$query)</code>
<code>axios.post(url, body)</code>	<code>Http::post(\$url, \$body)</code>
<code>headers: {...}</code>	<code>->withHeaders([...])</code>
<code>Authorization: Bearer</code>	<code>->withToken(\$t)</code>
<code>timeout: 5000</code>	<code>->timeout(5)</code>
<code>interceptors</code>	<code>middleware / global beforeSending</code>
<code>axios.all([...])</code>	<code>Http::pool(...)</code>
<code>throws on non-2xx (config)</code>	<code>->throw() / ->throwIf(...)</code>
<code>response .data</code>	<code>->json() / ->object() / ->collect()</code>

16.2 Response handling and error semantics

A key difference from Axios: by default the Laravel client **does not throw** on 4xx/5xx — you inspect the status or opt into throwing with `->throw()`. This is closer to `fetch` than Axios.

PHP

```
$response = Http::get($url);
$response->successful();    // 2xx
$response->failed();        // 4xx or 5xx
$response->clientError();   // 4xx
$response->serverError();   // 5xx
$response->status();        // int
$response->json('data.0.id'); // dot-path into the body

Http::get($url)->throw(); // throw RequestException on error
Http::get($url)->throwIf(fn ($r) => $r->status() === 419);
```

16.3 Retries, backoff, and timeouts

First-class retries are nicer than Axios (where you bolt on `axios-retry`).

PHP

```
Http::retry(3, 200, throw: false) // 3 attempts, 200ms base delay
    ->connectTimeout(3)->timeout(10)
    ->withToken($token)
    ->get($url);

// exponential backoff with a custom decision
Http::retry(5, fn (int $attempt) => $attempt * 250, function ($exception, $request) {
    return $exception instanceof ConnectionException; // retry only on connection
errors
})->get($url);
```

PRODUCTION TIP

Set **both** `connectTimeout` (time to establish the socket) and `timeout` (total time) on every outbound call — an external dependency without a timeout is how one slow vendor takes down your whole app by tying up FPM workers (Part 3). Retry only **idempotent** requests, and prefer jittered exponential backoff to avoid synchronizing retries into a stampede on the downstream service.

16.4 Concurrent requests

`Http::pool` fires requests concurrently via cURL multi — the within-request concurrency from Part 3, and the analog of `Promise.all`/`axios.all`.

PHP

```
$responses = Http::pool(fn (Pool $pool) => [
    $pool->as('users')->get('https://api.x.dev/users'),
    $pool->as('plans')->get('https://api.x.dev/plans'),
    $pool->as('flags')->get('https://api.x.dev/flags'),
]);
$users = $responses['users']->json();
```

16.5 Testing without hitting the network

This is where the Laravel client shines over Axios. `Http::fake()` intercepts outbound calls so tests never touch the network and you can assert on what was sent — no `nock/msw` setup.

PHP

```
Http::fake([
    'api.x.dev/users' => Http::response(['data' => [['id' => 1]]], 200),
    'api.x.dev/*'      => Http::response([], 404),
]);

// ... run code under test ...

Http::assertSent(fn ($request) =>
    $request->url() === 'https://api.x.dev/users' &&
    $request->hasHeader('Authorization'));
```

COMMON MISTAKE

Building an integration against a third party with no timeouts, no retries, and real network calls in the test suite (slow, flaky, rate-limited). Wrap external APIs behind a small client class, inject it, `Http::fake()` in tests, and always set timeouts + bounded retries. Treat every outbound call as a thing that *will* be slow or fail eventually.

WHEN NOT TO CALL SYNCHRONOUSLY

If the outbound call is not needed to build the HTTP response (sending to a CRM, posting to a webhook, syncing analytics), don't do it inline — dispatch a queued job (Part 13). A synchronous third-party call on the request path couples your latency and uptime to theirs.

EXERCISE 16.1

Wrap a third-party API in a `WeatherClient` service with `withToken`, `connectTimeout(3)`, `timeout(8)`, and `retry(3, backoff)`. Add a method that fetches three cities concurrently with `Http::pool`. Write a test using `Http::fake()` that asserts the token header was sent and that a 500 triggers retries. Then move a non-critical "log to analytics" POST into a queued job.

Part 17 · Files & Object Storage (S3)

File handling in Node means juggling `multer` for uploads, the AWS SDK for S3, `fs` for local disks, and signed-URL helpers. Laravel unifies all of this behind one **Storage** abstraction (Flysystem): the same API works against the local disk, S3, or any driver, so you develop against local files and deploy against S3 by changing one config value.

17.1 The disk abstraction

PHP

```
// config/filesystems.php defines "disks": local, public, s3, ...
use Illuminate\Support\Facades\Storage;

Storage::disk('s3')->put('reports/jan.pdf', $contents);
Storage::disk('local')->get('notes.txt');
Storage::disk('s3')->exists('reports/jan.pdf');
Storage::disk('s3')->delete('reports/jan.pdf');
$url = Storage::disk('s3')->url('reports/jan.pdf'); // public URL
```

JAVASCRIPT / NODE.JS

```
// Node equivalent: different SDKs per target, wired by hand
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3";
await s3.send(new PutObjectCommand({ Bucket, Key, Body }));
```

Task	Node	Laravel Storage
Upload middleware	<code>multer</code>	<code>\$request->file('x')</code> (built in)
Put object	AWS SDK <code>PutObject</code>	<code>Storage::put(\$path, \$data)</code>
Local disk	<code>fs.writeFile</code>	<code>Storage::disk('local')->put(...)</code>
Public URL	construct manually	<code>Storage::url(\$path)</code>
Signed URL	<code>getSignedUrl</code>	<code>Storage::temporaryUrl(\$path, \$expiry)</code>
Stream large file	<code>fs.createReadStream</code>	<code>Storage::readStream(\$path)</code>

17.2 Handling uploads safely

PHP

```
public function store(Request $request): JsonResponse
{
    $request->validate([
        'avatar' => ['required', 'file', 'mimes:jpg,jpeg,png,webp', 'max:2048'], //
```

```

KB
    });

    // store() generates a random, safe filename; never trust the client's name
    $path = $request->file('avatar')->store('avatars', 's3'); // returns the
key
    $request->user()->update(['avatar_path' => $path]);

    return response()->json(['path' => $path], 201);
}

```

Security essentials (Part 11 recap): validate MIME and size, let Storage generate the filename, keep user uploads on a disk/prefix where PHP is never executed, and put anything sensitive on a **private** disk.

17.3 Private files and temporary (signed) URLs

Never make private user files publicly readable. Store them on a private S3 bucket and hand clients **temporary signed URLs** that expire — the browser downloads directly from S3, and your app never proxies the bytes.

PHP

```

// Generate a URL valid for 5 minutes, then it stops working
$url = Storage::disk('s3')->temporaryUrl(
    $document->path,
    now()->addMinutes(5),
    ['ResponseContentDisposition' => 'attachment; filename="invoice.pdf"'],
);

```

JAVASCRIPT / NODE.JS

```

// Node equivalent
const url = await getSignedUrl(s3, new GetObjectCommand({ Bucket, Key }), {
  expiresIn: 300 });

```

NODE.JS MENTAL MODEL

`temporaryUrl()` is exactly `getSignedUrl` from the AWS SDK — a time-limited, signed link to a private object — but driver-agnostic, so the same call works for the local `public` disk (via a signed route) in development and real S3 in production. Offload the download to S3; do not stream large private files through PHP-FPM workers (Part 3).

17.4 Streaming and large files

For big files, stream rather than loading into memory (Part 2's lazy-evaluation lesson applied to I/O):

PHP

```
return Storage::disk('s3')->download($path, 'export.csv'); // streamed download
$stream = Storage::readStream($path); // read incrementally
Storage::writeStream('big/copy.csv', $stream); // write incrementally
```

17.5 Image and file processing

Image manipulation (resize, crop, optimize) uses the `intervention/image` package; heavy processing belongs on a **queue** (Part 13), not the request:

PHP

```
final class GenerateThumbnail implements ShouldQueue
{
    public function __construct(public readonly string $path) {}
    public function handle(): void
    {
        $img = Image::read(Storage::disk('s3')->get($this->path))->scale(width: 400);
        Storage::disk('s3')->put(Str::replaceLast('.', '_thumb.', $this->path),
(string) $img->encode());
    }
}
```

COMMON MISTAKE

Resizing images or generating PDFs synchronously inside the upload request. A few large images will exhaust FPM workers and time out. Accept the upload, store the original, dispatch a queued job for processing, and return immediately — then notify the client (broadcast/polling) when derivatives are ready.

EXERCISE 17.1

Build an avatar upload: validate type/size, store on `s3` with a generated name, save the key on the user, and dispatch a `GenerateThumbnail` job. Serve private documents via `temporaryUrl()` expiring in 5 minutes and prove the link 403s after expiry. Confirm the local `public` disk produces working signed URLs in development with the same code path.

Part 18 · Testing

You test Node with Jest (unit), Supertest (HTTP), and Testing Library. Laravel's testing story is arguably *stronger* out of the box: **Pest** (a delightful, Jest-like syntax over PHPUnit), a real HTTP testing client built in, database transactions that roll back after each test, factories for data, and one-line "fakes" for queues, mail, events, notifications, storage, and HTTP.

18.1 The tooling map

Node	Laravel
Jest / Vitest	Pest (or PHPUnit directly)
<code>describe / it / expect</code>	<code>describe / it / expect</code> (Pest)
Supertest (<code>request(app).get()</code>)	<code>\$this->getJson('/...')</code> (built in)
Testing Library	(N/A — <i>backend</i>)
<code>jest.fn()</code> / <code>jest.mock()</code>	<code>Mockery</code> , <code>Http::fake()</code> , facade fakes
test DB setup/teardown	<code>RefreshDatabase</code> (transaction rollback)
fixtures / factories	model factories (Part 8)
<code>nock</code> / <code>msw</code>	<code>Http::fake()</code>
fake timers	<code>travel()</code> / <code>Carbon::setTestNow()</code>

18.2 Feature tests: the highest-value tests you can write

A **feature test** boots the framework, hits a real route, and asserts on the JSON and the database — Supertest + a real DB, but faster because each test runs in a transaction that rolls back.

PHP

```
use function Pest\Laravel\{getJson, postJson, actingAs};

uses(Illuminate\Foundation\Testing\RefreshDatabase::class); // rollback per test

it('creates a project for the authenticated user', function () {
    $user = User::factory()->create();

    $response = actingAs($user)->postJson('/api/projects', [
        'name' => 'Launch',
    ]);

    $response->assertCreated() // 201
        ->assertJsonPath('data.name', 'Launch');
```

```

    expect(Project::where('owner_id', $user->id)->count()->toBe(1);
    $this->assertDatabaseHas('projects', ['name' => 'Launch', 'owner_id' => $user-
>id]);
});

it('rejects invalid input with 422', function () {
    actingAs(User::factory()->create())
        ->postJson('/api/projects', ['name' => ''])
        ->assertStatus(422)
        ->assertJsonValidationErrorFor('name');
});

it('forbids editing another users project', function () {
    $project = Project::factory()->create(); // owned by someone
else
    actingAs(User::factory()->create())
        ->putJson("/api/projects/{$project->id}", ['name' => 'Hijack'])
        ->assertForbidden(); // 403 (Policy, Part
10)
});

```

JAVASCRIPT / NODE.JS

```

// Supertest equivalent
it("creates a project", async () => {
    const res = await request(app).post("/api/projects")
        .set("Authorization", `Bearer ${token}`).send({ name: "Launch" });
    expect(res.status).toBe(201);
});

```

NODE.JS MENTAL MODEL

`RefreshDatabase` wraps each test in a DB transaction and rolls it back at the end, so tests are isolated and fast without truncating tables — the clean-database-per-test discipline you fight for in Node comes free. `actingAs` replaces manually minting a token for every request.

18.3 Faking the outside world

One-line fakes replace whole categories of Node mocking libraries, and let you **assert** that the right side effect was triggered without performing it.

PHP

```

use Illuminate\Support\Facades\{Queue, Mail, Event, Notification, Storage, Bus};

Queue::fake();
// ... hit the endpoint ...
Queue::assertPushed(SendWelcomeEmail::class); // job was dispatched, not run

Mail::fake();

```

```
Mail::assertSent(WelcomeMail::class, fn ($m) => $m->hasTo($user->email));

Notification::fake();
Notification::assertSentTo($user, InvoicePaid::class);

Event::fake([ProjectCreated::class]);
Event::assertDispatched(ProjectCreated::class);

Storage::fake('s3');
Storage::disk('s3')->assertExists("avatars/{$name}");

Http::fake(); // Part 16
Http::assertSent(fn ($r) => $r->url() === 'https://api.x.dev/users');
```

18.4 Unit tests, mocking, and time

Pure logic (a `Money` value object, a pricing rule) gets fast **unit tests** with no framework boot. For dependencies you don't want to hit, use `Mockery` via the container; for time-dependent logic, travel through time:

PHP

```
it('computes tax', function () {
    expect((new Pricing)->withVat(100, 0.15))->toBe(115.0);
});

it('expires tokens after 30 days', function () {
    $token = createToken();
    $this->travel(31)->days(); // fake timers
    expect($token->fresh()->isExpired()->toBeTrue());
});

// mock a collaborator resolved from the container
$this->mock(PaymentGateway::class)
    ->shouldReceive('charge')->once()->andReturn('ch_123');
```

18.5 What to test, and how much

Layer	Test type	Priority
Endpoints (auth, validation, happy path, 403/404)	Feature	highest
Business logic (Actions, value objects)	Unit	high
Authorization (Policies)	Feature/Unit	high
Queued jobs (dispatched + handle logic)	Feature + Unit	medium
External clients	Unit with <code>Http::fake()</code>	medium
Framework glue (Resources, casts)	light Feature coverage	low

PRODUCTION TIP

Bias toward **feature tests** that exercise a route end-to-end: they cover routing, validation, authorization, the controller, the model, and the response in one test, and they survive refactors of the internals. A suite of these is what lets you extract Actions and rework queries fearlessly. Run them in CI with a Postgres/MySQL service, not SQLite, if you use database-specific features.

EXERCISE 18.1

Cover your Projects+Tasks API with feature tests: create (201), validation (422), unauthorized edit (403), missing record (404), and pagination shape. Add `Queue::fake()` to assert a welcome job is dispatched on registration without running it, and a unit test for one value object. Aim for the suite to run in under a couple of seconds via `RefreshDatabase`.

Part 19 · Artisan — The CLI

Artisan is Laravel's command-line tool. It is your `npm scripts`, your code generators, your database console, your REPL, and your operational toolbox in one binary. If you've used the NestJS CLI or Rails' `bin/rails`, Artisan is that — but broader.

19.1 The commands you'll live in

Task	npm / Node habit	Artisan
REPL	<code>node</code>	<code>php artisan tinker</code>
Generate scaffolding	<code>plop</code> / by hand	<code>php artisan make:*</code>
Run migrations	<code>Prisma migrate</code>	<code>php artisan migrate</code>
List routes	<code>print router</code>	<code>php artisan route:list</code>
Run queue worker	<code>node worker.js</code>	<code>php artisan queue:work</code>
Run scheduler	<code>node-cron process</code>	<code>php artisan schedule:run</code>
Clear caches	<code>rm -rf</code>	<code>php artisan optimize:clear</code>
Custom scripts	<code>npm run x</code>	<code>php artisan app:my-command</code>

SHELL

```
php artisan make:model Project -mfc --api  # model + migration + factory +
controller
php artisan make:request StoreProjectRequest
php artisan make:policy ProjectPolicy --model=Project
php artisan make:job SendWelcomeEmail
php artisan make:command PruneExports
php artisan migrate --seed
php artisan route:list --path=api
php artisan optimize                    # cache config+routes+events (prod)
php artisan optimize:clear              # drop all caches (after deploy/changes)
```

19.2 Tinker — the REPL

`php artisan tinker` boots your full application into an interactive shell, so you can run real Eloquent queries, dispatch jobs, and poke at services — like a `node` REPL with your models and container already loaded.

PHP

```
>>> User::factory()->create()
>>> Project::with('tasks')->find(1)->tasks->pluck('title')
```

```
>>> SendWelcomeEmail::dispatch(User::first())
>>> Cache::put('x', 1, 60); Cache::get('x')
```

PRODUCTION TIP

Tinker is invaluable for debugging and one-off data fixes on a staging box. On production, prefer a **written, reviewed Artisan command** over ad-hoc Tinker mutations — a command is repeatable, testable, logged, and can be dry-run. Treat production Tinker like `psql` on prod: allowed, but with extreme care.

19.3 Writing custom commands

Custom commands are the right home for operational scripts (backfills, imports, maintenance, report generation) and the things you schedule (Part 14). They get dependency injection, argument/option parsing, and rich console output for free.

PHP

```
final class PruneExports extends Command
{
    protected $signature = 'exports:prune {--days=30} {--dry-run}';
    protected $description = 'Delete export files older than N days';

    public function handle(): int
    {
        $days = (int) $this->option('days');
        $cutoff = now()->subDays($days);
        $stale = Export::where('created_at', '<', $cutoff)->get();

        $this->info("Found {$stale->count()} exports older than {$days} days.");
        if ($this->option('dry-run')) return self::SUCCESS;

        $bar = $this->output->createProgressBar($stale->count());
        foreach ($stale as $export) {
            Storage::disk('s3')->delete($export->path);
            $export->delete();
            $bar->advance();
        }
        $bar->finish();
        return self::SUCCESS;                                     // exit code (0 = ok)
    }
}
```

SHELL

```
php artisan exports:prune --days=60 --dry-run
```

JAVASCRIPT / NODE.JS

```
// Node equivalent: a bin script wired into package.json + a CLI parser
// "scripts": { "prune": "node scripts/prune.js" }, using commander/yargs
```

NODE.JS MENTAL MODEL

A custom Artisan command is your `scripts/prune.js` with batteries: argument parsing (like `commander`), DI of services, progress bars/tables, an exit code, and — because it's a first-class command — it's directly schedulable (`Schedule::command('exports:prune')`) and testable (`$this->artisan('exports:prune')->assertExitCode(0)`).

19.4 The Process API — running external commands

When you genuinely need to shell out (`ffmpeg`, a Python script, `git`), Laravel's **Process** facade is the ergonomic wrapper over `proc_open` — the analog of Node's `child_process`.

PHP

```
use Illuminate\Support\Facades\Process;

$result = Process::timeout(60)->run('ffmpeg -i in.mp4 out.webm');
if ($result->failed()) { report(new RuntimeException($result->errorOutput())); }

// concurrent processes (like Promise.all over child_process)
$results = Process::pool(fn ($pool) => [
    $pool->command('git fetch'),
    $pool->command('npm run build'),
])->start()->wait();
```

Node <code>child_process</code>	Laravel Process
<code>exec</code> / <code>execSync</code>	<code>Process::run(...)</code>
<code>spawn</code> (streaming)	<code>Process::start(...)</code> + <code>->wait()</code>
<code>stdout</code> / <code>stderr</code>	<code>->output()</code> / <code>->errorOutput()</code>
exit code	<code>->exitCode()</code> / <code>->successful()</code>
parallel	<code>Process::pool(...)</code>

EXERCISE 19.1

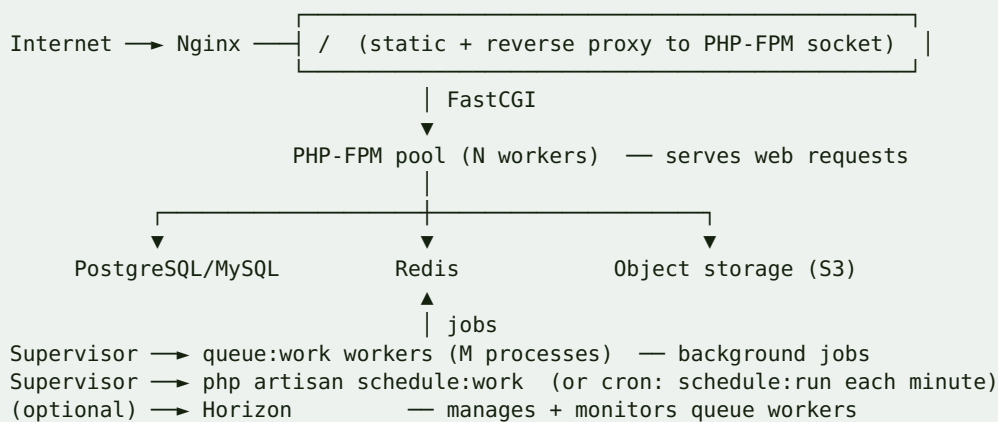
Write an `app:import-users {file} {--dry-run}` command that streams a CSV (use a `LazyCollection`, Part 2), validates each row, and creates users in chunks, printing a progress bar and a summary table. Add a feature test with `$this->artisan('app:import-users', ...)->assertExitCode(0)`. Schedule a `--dry-run` nightly and log the count.

Part 20 · Production Deployment

Deploying Node is often "run the process, put a reverse proxy in front, keep it alive with PM2." Deploying Laravel means orchestrating a small **fleet** (Part 3): Nginx → PHP-FPM for web requests, one or more `queue:work` workers, a cron line for the scheduler, plus Postgres/MySQL and Redis. This part is the production checklist and the reference topology.

20.1 The production topology

DIAGRAM



Role	Node analog	PHP/Laravel
Web process	<code>node server.js</code> under PM2	Nginx + PHP-FPM
Process supervisor	PM2	Supervisor (workers) / systemd
Background workers	BullMQ workers	<code>queue:work</code> under Supervisor / Horizon
Scheduler	node-cron in-process	cron → <code>schedule:run</code>
Reverse proxy	Nginx	Nginx

20.2 The Nginx + PHP-FPM front

Nginx serves static files and forwards PHP to FPM over a socket. The document root is `public/` (never the project root — that would expose `.env` and code).

NGINX

```

server {
    listen 80;
    root /var/www/app/public;           # public/ only
    index index.php;

    location / { try_files $uri $uri/ /index.php?$query_string; }
  
```



```

location ~ /\.php$ {
    fastcgi_pass unix:/var/run/php/php8.3-fpm.sock;
    fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
    include fastcgi_params;
}
location ~ /\.(!well-known).* { deny all; } # block dotfiles
}

```

20.3 Keeping workers alive with Supervisor

Workers are long-lived processes that must be auto-restarted if they die or are killed on deploy. Supervisor is the standard (PM2's role for PHP):

INI / .ENV

```

[program:app-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/app/artisan queue:work redis --sleep=3 --tries=3 --max-time=3600
autostart=true
autorestart=true
numprocs=4 ; run 4 worker processes
stopwaitsecs=3600 ; let a job finish before killing (graceful)
user=www-data

```

Or run **Horizon** (`php artisan horizon`, itself supervised) for auto-balanced workers and a live dashboard.

20.4 The deploy sequence (zero-downtime)

Order matters. The safe sequence:

SHELL

```

composer install --no-dev --optimize-autoloader # deps, prod autoloader
php artisan down --render="errors::503" --retry=5 # optional maintenance window
php artisan migrate --force # run migrations (see below)
php artisan optimize # cache config, routes, events,
views
php artisan queue:restart # workers pick up NEW code (Part
13!)
php artisan up # back online

```

Zero-downtime migrations are the deploy skill that most often bites Node developers new to a compiled-config framework. The rule is **expand** → **migrate** → **contract**, never a breaking change in one step:

1. **Expand**: add the new nullable column / new table (old code ignores it).
2. **Deploy** code that writes to both old and new.
3. **Backfill** existing rows via a queued job.

4. **Contract:** in a later deploy, switch reads to the new column, then drop the old one and add constraints.

COMMON MISTAKE

Three deploy-day classics: (1) forgetting `queue:restart`, so workers keep running old code (Part 13); (2) renaming/dropping a column in the same deploy that ships the code depending on it, causing errors during the rollout window; (3) leaving `env()` calls in app code that return `null` once `config:cache` runs (Part 5). Bake `optimize` and `queue:restart` into the deploy script and never skip them.

20.5 Config, secrets, and health checks

INI / .ENV

```
# .env (production) – never committed
APP_ENV=production
APP_DEBUG=false           # CRITICAL: never true in prod (leaks stack traces)
APP_KEY=base64:...        # stable across all servers
DB_CONNECTION=pgsql
QUEUE_CONNECTION=redis
CACHE_STORE=redis
SESSION_DRIVER=redis
```

Add a health endpoint (Laravel 11 ships `/up`) for your load balancer and orchestrator readiness checks. Ship logs as **structured JSON** to stdout/a log service (Part 22 in the Python book covers observability patterns that apply equally here); set `LOG_CHANNEL=stderr` in containers.

20.6 Docker

A production image typically uses PHP-FPM + a separate Nginx container (or **FrankenPHP**, which bundles a server and enables Octane). One image, run with different commands for web vs. worker vs. scheduler:

DOCKERFILE

```
FROM php:8.3-fpm-alpine
RUN docker-php-ext-install pdo_pgsql opcache bcmath && \
    apk add --no-cache $PHPIZE_DEPS && pecl install redis && docker-php-ext-enable
redis
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
WORKDIR /var/www/app
COPY composer.* ./
RUN composer install --no-dev --no-scripts --optimize-autoloader
COPY . .
RUN php artisan optimize
# web:          php-fpm
```

```
# worker:  php artisan queue:work  (or horizon)
# scheduler:php artisan schedule:work
```

PRODUCTION TIP

Turn on **OPcache** (Part 3) and run `php artisan optimize` in the image build/deploy — without them PHP recompiles and rebuilds the container graph on every request. Set `opcache.validate_timestamps=0` and rely on rebuilding the image (or clearing OPcache) per deploy. This single step is often a multiple-x throughput difference.

WHEN NOT TO REACH FOR KUBERNETES/OCTANE ON DAY ONE

A single VM with Nginx + FPM + Supervisor + managed Postgres/Redis runs a serious production app comfortably and is far easier to operate. Add Octane when throughput demands it, and orchestration when you genuinely need horizontal scale and rolling deploys — not because the architecture diagram looks more impressive. Complexity is a cost paid daily.

EXERCISE 20.1

Write a Docker Compose stack: `app` (FPM), `nginx`, `worker` (`queue:work`), `scheduler` (`schedule:work`), `postgres`, `redis`. Add a deploy script running the Part 20.4 sequence including `queue:restart`. Practice an expand→contract migration: add a nullable `archived_at`, backfill via a job, then enforce/observe it in a second deploy — with `/up` green throughout.

Part 21 · Advanced Topics

With the fundamentals in place, this part surveys the capabilities you reach for as an app matures: real-time features, full-text search, performance tuning, and the concurrency/Octane story pulled together. Each is deep enough for its own book; the goal here is to map it to what you know and point you at the right tool.

21.1 Real-time: broadcasting and WebSockets

In Node you'd hold WebSocket connections in a long-lived process (Socket.IO, `ws`) — impossible in classic PHP, where requests die (Part 3). Laravel's answer is **broadcasting**: your app fires events, and a **separate** WebSocket server delivers them to subscribed clients. The server can be **Reverb** (Laravel's first-party WebSocket server), Pusher, or Ably.

PHP

```
final class MessageSent implements ShouldBroadcast
{
    public function __construct(public readonly Message $message) {}
    public function broadcastOn(): Channel
    {
        return new PrivateChannel("conversation.{$this->message->conversation_id}");
    }
}
broadcast(new MessageSent($message)); // pushed to subscribers via Reverb/Pusher
```

JAVASCRIPT / NODE.JS

```
// Browser (Laravel Echo) – mirrors Socket.IO's client API
Echo.private(`conversation.${id}`).listen("MessageSent", (e) => render(e.message));
```

Socket.IO (Node)	Laravel Broadcasting
server holds connections in-process	separate server (Reverb/Pusher/Ably)
<code>io.to(room).emit(evt, data)</code>	<code>broadcast(new Event())</code> on a Channel
rooms	channels (public/private/presence)
client <code>socket.on</code>	<code>Echo.private(...).listen(...)</code>
auth on connection	channel authorization callbacks

NODE.JS MENTAL MODEL

The mental shift: your PHP app doesn't *hold* the socket — it *publishes* an event to a dedicated realtime server that holds sockets. This decouples realtime from your request/worker fleet and scales it independently. Reverb is a PHP process you run alongside FPM (Part 20); it is the one long-lived connection holder, playing the role your Node process played.

21.2 Full-text search: Scout

Laravel Scout is a driver-based search abstraction: mark a model `Searchable`, and it keeps an external index (Meilisearch, Algolia, or even a database/Postgres driver) in sync automatically.

PHP

```
final class Project extends Model {
    use Searchable;
    public function toSearchableArray(): array
    {
        return ['name' => $this->name, 'description' => $this->description];
    }
}
Project::search('launch plan')->paginate(20);    // hits the search engine
```

Index updates are queued, so writes stay fast. This is the equivalent of wiring your Node models to Elasticsearch/Meilisearch by hand, standardized.

21.3 Concurrency and Octane, consolidated

Part 3 introduced these; here is the practical decision layer:

Need	Tool	Notes
Concurrent outbound HTTP	<code>Http::pool</code>	within one request
Run independent callbacks in parallel	<code>Concurrency::run(...)</code>	separate procs
Offload slow work	Queues (Part 13)	the default answer
Higher request throughput	Octane	warm app in memory; reset state!
CPU-bound job	queue on dedicated workers / <code>Process</code>	keep off web tier
Realtime fan-out	Broadcasting + Reverb	separate server

COMMON MISTAKE

Adopting Octane for speed and then leaking state across requests via static properties, container singletons holding request data, or global mutation (Part 3). Under FPM these were harmless; under Octane they are cross-request data bleed. Audit singletons, avoid static mutable state, and use Octane's `flush`/listeners to reset per-request services.

21.4 Performance: the measurable wins, in order

Optimize in this order — earlier items dwarf later ones:

1. **Kill N+1 queries** (Part 7): eager-load; the single biggest win.
2. **Index the columns you filter/sort/join on** (Part 7): a missing index turns a 2 ms query into a 2 s table scan.
3. **OPcache + artisan optimize** (Parts 3, 20): compile once, cache config.
4. **Cache expensive reads** (Part 15): `remember` with stampede protection.
5. **Move slow work off the request** (Part 13): queues.
6. **Paginate / stream** large result sets (`cursor()` , `chunk()`): bound memory.
7. **Then** consider Octane for raw throughput.

PHP

```
// Find N+1s automatically in dev/test
Model::preventLazyLoading(! app()->isProduction()); // throws on lazy load

// Detect slow queries
DB::whenQueryingForLongerThan(500, fn ($conn) => Log::warning('slow query batch'));
```

PRODUCTION TIP

Instrument before you optimize. Laravel **Telescope** (local/staging) shows per-request query counts, timings, cache hits, and job activity; **Debugger** overlays the same in the browser. Ninety percent of "Laravel is slow" turns out to be an N+1 or a missing index, both visible in one glance at the query panel. Guessing at performance is how you optimize the wrong thing.

WHEN NOT TO OPTIMIZE

Don't add caching, Octane, or read replicas to an app that serves 20 requests a minute with 30 ms responses. Fix correctness and clarity first; optimize when a profiler (not a hunch) shows a real bottleneck under real load. Premature optimization buys complexity you pay for forever and speed you didn't need.

EXERCISE 21.1

Add broadcasting: fire a `TaskAssigned` event on a private channel and log the payload a client would receive. Make `Project` `Searchable` with the database driver and paginate a search. Then take a deliberately-slow endpoint ($N+1$ + missing index), profile it with Telescope, and cut its query count and time by $\geq 5\times$. Document each change and its measured effect.

Part 22 · Capstone Projects

Three production-oriented projects that force you to combine everything. Each gives you an architecture, a database schema, the route surface, the key models and logic, and the production concerns (auth, validation, queues, caching, tests, deployment). Build them in order; each raises the bar.

22.1 Project A — Task Management API

A multi-user task manager: users own **projects**, projects have **tasks**, tasks are assigned to members, and activity is tracked. This consolidates Parts 5–12, 18.

Domain and schema

DIAGRAM

```
users      (id, name, email, password, created_at)
projects   (id, owner_id→users, name, status[enum], archived_at?, timestamps)
project_user (project_id, user_id, role[owner|member])      -- pivot, many-to-many
tasks      (id, project_id→projects, assignee_id?→users, title,
            status[todo|doing|done], priority[int], due_at?, timestamps, deleted_at?)
activities (id, subject_type, subject_id, causer_id→users, event, timestamps) -- polymorphic
```

Route surface

PHP

```
Route::middleware('auth:sanctum')->group(function () {
    Route::apiResource('projects', ProjectController::class);
    Route::apiResource('projects.tasks', TaskController::class)->scoped();
    Route::post('projects/{project}/members', [MemberController::class, 'store']);
    Route::post('tasks/{task}/assign', AssignTaskController::class); // invokable
    Route::patch('tasks/{task}/status', [TaskController::class, 'updateStatus']);
});
```

Key building blocks

- **Authorization:** `ProjectPolicy` (owner/member) and `TaskPolicy` (must be a project member) enforced in Form Requests and controllers (Part 10). Scoped bindings ensure a task belongs to its project (Part 6).
- **Validation:** `StoreTaskRequest` validates `status` against the enum, `assignee_id` must be a project member (custom rule).
- **Events + queue:** `TaskAssigned` event → **queued** listener emails the assignee and writes an `activities` row (Part 12, 13). Notifications never block the request.

- **Caching:** per-project task counts cached with `remember`, invalidated by a model observer on task write (Part 15).
- **Serialization:** `TaskResource` includes the assignee only when loaded (`whenLoaded`) to avoid N+1 (Part 8).

Production concerns

Eager-load `project.members` and `assignee` on list endpoints; index `(project_id, status)` and `assignee_id`; paginate everything; feature-test the 403/404/422 paths and the "assigning a non-member is rejected" rule; run `queue:work` for the notification/activity jobs.

BUILD A

Implement A end to end with ≥ 15 feature tests. Prove: a member can create tasks but only an owner can archive a project; assigning to a non-member returns 422; the assignee email is *dispatched* (not sent) under `Queue::fake()`; and the project task-count cache is busted on task create.

22.2 Project B — E-commerce API

Adds money, inventory, transactions, and idempotency — where correctness under concurrency matters. Consolidates Parts 7, 9, 11, 13, 15.

Domain and schema

DIAGRAM

```
products      (id, name, slug[unique], price_cents, currency, active, timestamps)
inventories   (id, product_id→products, quantity, version[int]) -- optimistic lock
carts         (id, user_id→users, session_token?, timestamps)
cart_items    (id, cart_id→carts, product_id→products, quantity, unit_price_cents)
orders        (id, user_id→users, status[pending|paid|failed|refunded],
               total_cents, currency, idempotency_key?[unique], timestamps)
order_items   (id, order_id→orders, product_id, quantity, unit_price_cents)
payments      (id, order_id→orders, provider, provider_charge_id, status, timestamps)
```

The critical path: placing an order

The checkout must be **atomic**, **idempotent**, and **inventory-safe** — exactly the concurrency lessons from Parts 7, 9, and 13.

PHP

```
final class PlaceOrder
{
    public function handle(User $user, Cart $cart, string $idempotencyKey): Order
    {
        // idempotency: replay if this key already produced an order (Part 9)
        if ($existing = Order::where('idempotency_key', $idempotencyKey)->first()) {
            return $existing;
        }
    }
}
```

```

    return DB::transaction(function () use ($user, $cart, $idempotencyKey) {
        $order = Order::create([
            'user_id' => $user->id, 'status' => 'pending',
            'idempotency_key' => $idempotencyKey, 'currency' => 'USD',
            'total_cents' => 0,
        ]);
        $total = 0;
        foreach ($cart->items as $item) {
            // pessimistic lock the inventory row so two orders can't oversell
            (Part 7)
                $inv = Inventory::where('product_id', $item->product_id)
                    ->lockForUpdate()->firstOrFail();
                if ($inv->quantity < $item->quantity) {
                    throw new InsufficientStock($item->product_id); // -> 409 (Part
9)
                }
                $inv->decrement('quantity', $item->quantity);
                $order->items()->create([
                    'product_id' => $item->product_id,
                    'quantity' => $item->quantity,
                    'unit_price_cents' => $item->unit_price_cents,
                ]);
                $total += $item->quantity * $item->unit_price_cents;
            }
            $order->update(['total_cents' => $total]);
            ChargePayment::dispatch($order); // charge async on a worker
            (Part 13)
                return $order;
        });
    }
}

```

Key building blocks

- **Money as integer cents** everywhere (never floats); a `Money` value object (Part 12) for arithmetic and formatting.
- **Idempotency middleware** on `POST /orders` plus the DB-level unique `idempotency_key` as a backstop (Part 9).
- **Payment** via a `PaymentGateway` contract with a `FakeGateway` for tests and `StripeGateway` in prod (Part 5); charging is a queued, idempotent job (Part 13) that updates the order and emits `OrderPaid`.
- **Inbound webhooks** from the payment provider verify HMAC signatures (Part 11), dedupe by event id, and reconcile order status (Part 9).

Production concerns

Wrap the whole checkout in a transaction with row locks; test the oversell race (two concurrent orders for the last unit — one succeeds, one 409s); ensure the charge job is idempotent across retries; cache the product catalog with `flexible()` (Part 15); rate-limit checkout.

BUILD B

Implement checkout with pessimistic inventory locking and idempotent ordering. Write a concurrency test that fires two orders for the last unit and asserts exactly one succeeds. Verify a retried `POST /orders` with the same idempotency key creates one order. Simulate a payment webhook and assert the order flips to `paid` exactly once even if delivered twice.

22.3 Project C — Webhook / Integration Platform

An inbound/outbound webhook hub: receive events from providers, transform them, and deliver them to customer endpoints with retries and signatures. This is the queue/reliability capstone — Parts 3, 11, 13, 16.

Domain and schema

DIAGRAM

```
sources      (id, name, signing_secret, timestamps)      -- inbound providers
inbound_events (id, source_id→sources, external_id[unique per source],
               payload[json], received_at, processed_at?) -- dedupe by external_id
endpoints     (id, user_id→users, url, signing_secret, active, timestamps) -- customer sinks
subscriptions (id, endpoint_id→endpoints, event_type)    -- what each endpoint wants
deliveries     (id, endpoint_id→endpoints, inbound_event_id→inbound_events,
               status[pending|delivered|failed], attempts, last_status_code,
               next_attempt_at?, timestamps)             -- outbound attempts
```

The two pipelines

Inbound (receive fast, process async — Part 9):

PHP

```
public function receive(string $source, Request $request): Response
{
    $src = Source::where('name', $source)->firstOrFail();
    // 1. verify signature against RAW body (Part 11)
    $expected = hash_hmac('sha256', $request->getContent(), $src->signing_secret);
    abort_unless(hash_equals($expected, $request->header('X-Signature', '')), 400);

    // 2. dedupe (at-least-once delivery) and persist
    $event = InboundEvent::firstOrCreate(
        ['source_id' => $src->id, 'external_id' => $request->input('id')],
        ['payload' => $request->all(), 'received_at' => now()],
    );
    // 3. fan out to matching subscriptions on a worker, return fast
    if (! $event->wasRecentlyCreated) return response()->noContent(); // already
    have it
    FanOutEvent::dispatch($event);
    return response()->noContent(); // 204
    immediately
}
```

Outbound (deliver with retries, backoff, and signatures — Parts 13, 16):

PHP

```

final class DeliverWebhook implements ShouldQueue
{
    public int $tries = 8;
    public function __construct(public readonly Delivery $delivery) {}
    public function backoff(): array { return [10, 30, 60, 300, 900, 1800, 3600]; }

    public function handle(): void
    {
        $d = $this->delivery->fresh();
        if ($d->status === 'delivered') return; // idempotent
        $body = $d->inboundEvent->payload;
        $sig = hash_hmac('sha256', json_encode($body), $d->endpoint->
        >signing_secret);

        $res = Http::withHeaders(['X-Signature' => $sig])
            ->timeout(10)->connectTimeout(3)
            ->post($d->endpoint->url, $body); // (Part 16)

        $d->increment('attempts');
        if ($res->successful()) {
            $d->update(['status' => 'delivered', 'last_status_code' => $res-
            >status()]);
        } else {
            $d->update(['status' => 'failed', 'last_status_code' => $res->status()]);
            throw new DeliveryFailed(); // triggers retry with backoff until
        }
    }
}

```

Production concerns

Everything hinges on the Part 3 model: receive on the web tier in milliseconds, do all real work on **workers**; dedupe inbound by `(source, external_id)`; **sign** outbound and let customers verify; make delivery idempotent so retries never double-deliver; expose a **Horizon** dashboard (Part 13) and a deliveries admin view; alert on deliveries stuck in `failed` past all retries (a dead-letter). Rate-limit inbound per source; cap payload size.

BUILD C

Implement both pipelines. Test: an inbound event delivered twice is stored once (dedupe); a flaky customer endpoint (500 then 200) is retried with backoff and ends `delivered` with the right attempt count; a tampered inbound signature is rejected 400; outbound requests carry a valid `X-Signature` a customer can verify. Run it under Horizon and confirm the web endpoint returns 204 in well under 50 ms regardless of downstream speed.

YOU ARE NOW PRODUCTION-READY

If you can build all three — a permissioned CRUD platform, a concurrency-correct commerce flow, and a reliable async delivery system — you can build essentially any Laravel backend. The remaining growth is depth: domain modeling, operational maturity, and reading the framework's source when the docs run out.

Part 23 · Master Cheat Sheet & Closing Roadmap

Bookmark this part. It compresses the whole book into lookup tables: **Node.js** → **PHP** → **Laravel** for every subsystem, plus the closing roadmap.

23.1 Tooling & ecosystem

Node.js	PHP	Laravel
<code>npm</code> / <code>pnpm</code>	Composer	Composer
<code>package.json</code>	<code>composer.json</code>	<code>composer.json</code>
<code>package-lock.json</code>	<code>composer.lock</code>	<code>composer.lock</code>
<code>node_modules/</code>	<code>vendor/</code>	<code>vendor/</code>
<code>npm ci</code>	<code>composer install</code>	<code>composer install</code>
<code>npx <bin></code>	<code>vendor/bin/<bin></code>	<code>php artisan ...</code>
<code>npm run <script></code>	<code>composer run <script></code>	<code>php artisan <cmd></code>
ESLint / Prettier	PHP-CS-Fixer / Pint	Pint
tsc (type check)	PHPStan / Psalm	Larastan (PHPStan)
<code>.env</code> + dotenv	<code>.env</code> + getenv	<code>.env</code> → <code>config()</code>
node REPL	<code>php -a</code>	<code>php artisan tinker</code>

23.2 Language constructs

JavaScript / TypeScript	PHP
<code>const x = 1</code>	<code>\$x = 1;</code>
<code>`Hi \${name}`</code>	<code>"Hi \${name}"</code>
<code>a + b</code> (strings)	<code>\$a . \$b</code>
<code>{ a: 1 }</code> object	<code>['a' => 1]</code> associative array
<code>[1, 2, 3]</code> array	<code>[1, 2, 3]</code> list array
<code>obj.prop</code>	<code>\$obj->prop</code>
<code>Class.staticMethod()</code>	<code>Class::staticMethod()</code>
<code>x === y</code>	<code>\$x === \$y</code>
<code>x ?? y</code>	<code>\$x ?? \$y</code>
<code>a?.b?.c</code>	<code>\$a?->b?->c</code>

JavaScript / TypeScript	PHP
<code>arr.map(fn)</code>	<code>array_map(\$fn, \$arr) / collect(\$arr)->map(\$fn)</code>
<code>arr.filter(fn)</code>	<code>array_filter(\$arr, \$fn) / ->filter(\$fn)</code>
<code>switch</code>	<code>match(...)</code> (strict, returns value)
<code>(x) => x*2</code>	<code>fn(\$x) => \$x*2</code>
<code>function(){ }</code> w/ closure	<code>function() use (\$v) { }</code>
<code>...rest / ...spread</code>	<code>...\$rest / ...\$spread</code>
named via options object	native named args: <code>f(age: 3)</code>
<code>enum</code>	<code>enum Status: string { case A = 'a'; }</code>
<code>interface / abstract</code>	<code>interface / abstract class</code>
mixin	<code>trait + use</code>
<code>import / export</code>	<code>namespace + use</code> (PSR-4 autoload)
<code>try/catch(e)</code>	<code>try/catch(TypedException \$e)</code>
<code>function* / yield</code>	<code>function ...(): Generator { yield }</code>
decorators	<code>attributes #[...]</code>

23.3 Framework & HTTP

Express / Node	Laravel
<code>express.Router()</code>	<code>Route:: / routes/api.php</code>
<code>router.get('/x/:id', h)</code>	<code>Route::get('/x/{id}', \$action)</code>
<code>req.params.id</code>	method arg <code>\$id</code>
<code>req.query.q</code>	<code>\$request->query('q')</code>
<code>req.body</code>	<code>\$request->validated() / ->input()</code>
<code>res.json(x)</code>	<code>response()->json(\$x) / a Resource</code>
<code>res.status(201)</code>	<code>->response()->setStatusCode(201)</code>
<code>middleware (req,res,next)</code>	<code>Middleware handle(\$r, \$next)</code>
CRUD routes by hand	<code>Route::apiResource(...)</code>
manual fetch + 404	route model binding + <code>findOrFail</code>
Zod / Joi	Form Request / <code>Validator</code>
DTO / serializer	API Resource
DI by hand	service container (autowiring)
app singletons	<code>singleton()</code> bindings / providers

23.4 Data, jobs, and infrastructure

Node.js	PHP / Laravel
Prisma	Eloquent (Active Record)
<code>prisma.x.findMany</code>	<code>X::where(...)->get()</code>
<code>include: {...}</code>	<code>->with([...])</code> (eager load)
Prisma migrate	Artisan migrations
<code>\$transaction</code>	<code>DB::transaction(fn)</code>
<code>SELECT ... FOR UPDATE</code>	<code>->lockForUpdate()</code>
Axios / fetch	<code>Http::</code> client
<code>Promise.all</code>	<code>Http::pool</code> / <code>Concurrency::run</code>
BullMQ	Laravel Queues
<code>queue.add(job)</code>	<code>Job::dispatch(...)</code>
BullMQ worker	<code>php artisan queue:work</code>
Bull Board	Horizon
node-cron	Laravel Scheduler (<code>schedule:run</code>)
Redis (ioredis)	<code>Cache::</code> / <code>Redis::</code>
in-process cache <code>Map</code>	external cache (Redis) — no in-proc!
<code>getSignedUrl</code> (S3)	<code>Storage::temporaryUrl(...)</code>
multer	<code>\$request->file(...)</code>
Socket.IO	Broadcasting + Reverb/Pusher
<code>child_process</code>	<code>Process::</code>
Jest / Vitest	Pest / PHPUnit
Supertest	<code>\$this->getJson(...)</code>
nock / msw	<code>Http::fake()</code>
PM2	Supervisor / Horizon / systemd
<code>node server.js</code>	Nginx + PHP-FPM (or Octane)
bcrypt / argon2	<code>Hash::</code>
jsonwebtoken / Passport	Sanctum (tokens) / Passport (OAuth2)
custom RBAC	<code>spatie/laravel-permission</code>
Elasticsearch client	Scout (Meilisearch/Algolia)

23.5 The runtime, in one table

Concept	Node.js	Classic PHP	Laravel Octane
Process	one, long-lived	one per request	one, long-lived
Concurrency	event loop	many FPM workers	worker holds warm app
Memory across requests	shared	wiped	shared (reset carefully!)
I/O	non-blocking	blocking	blocking (per worker)
Background work	in-process ok	queues/workers	queues/workers
State leak risk	real	none	real
New code visible	on restart	instantly	on reload/restart
Worker code visible	—	on <code>queue:restart</code>	on <code>queue:restart</code>

23.6 Closing Roadmap — from here to fluent

You have the map; here is the deliberate-practice plan to internalize it.

Weeks 1-2 · Language & FP (Parts 1-2). Port 20+ Node utilities to PHP. Rebuild one data-transform module with Collections. Turn on `strict_types` and Larastan; fix every type error. *Checkpoint:* you stop translating in your head.

Week 3 · Runtime (Part 3). Diagram your Express apps and their FPM equivalents. Do the `Http::pool` latency experiment. Explain shared-nothing and Octane's tradeoff to a colleague. *Checkpoint:* you know *where* work runs.

Weeks 4-5 · Framework & data (Parts 4-8). Scaffold an app; build Users/Projects/Tasks CRUD with relationships, Form Requests, Resources, and zero N+1s. *Checkpoint:* you build a resource without looking up the steps.

Week 6 · APIs, auth, security (Parts 9-11). Add pagination/filtering, Sanctum, Policies, rate limiting, and the security checklist. *Checkpoint:* your API is safe to expose publicly.

Week 7 · Async & operations (Parts 13-17, 19). Move slow work to queues, add a scheduled command, cache a hot endpoint, wrap a third-party API, handle file uploads to S3. *Checkpoint:* nothing slow runs on the request path.

Week 8 · Testing & deploy (Parts 18, 20). Feature-test your API; containerize the full fleet; practice an expand→contract migration with `queue:restart`. *Checkpoint:* you can ship to production with confidence.

Weeks 9-12 · Capstones (Part 22). Build all three. *Checkpoint:* you are an independent, production-grade Laravel backend developer.

Then keep going: read the framework source when docs run out, adopt Octane and Reverb when you need them, and deepen your domain modeling (Part 12) as apps grow. The ecosystem is deep, but you now hold the thread that connects all of it back to the Node you already know. Welcome to PHP and Laravel.